ACADEMIC DBMS GROUP PROJECT REPORT

SELF-HEALING RELATIONAL DATABASE INTEGRITY SYSTEM

Autonomous Anomaly Detection, Relational DAG Modeling, Transaction-Safe Remediation, and Traceability Engine for MySQL InnoDB

Attribute	Specification Details
Implementation Version	Release 2.4.0 (Enterprise Academic Edition)
Database Management System	MySQL 8.0+ / MariaDB 10.4+ (InnoDB Engine via local XAMPP)
Target Database Instance	self_healing_test_db (16 Relational & Engine Tables)
Backend Web Services	Python 3.13, FastAPI 0.110+, PyMySQL, NetworkX 3.2+
Frontend Presentation	React 18.3, Vite 5.4, Tailwind CSS 3.4, Recharts, Lucide Icons
Concurrency Safeguards	ACID Transactions (BEGIN, COMMIT, ROLLBACK, SAVEPOINT), SHA-256 Tokens
Audit & Immutability	Trigger-Protected Write-Once Audit Logs, Sensitive Redaction
Verification Status	Automated Pytest Suite (39 Tests Passing, 0 Failures)
Target Audience Scope	Dual-Level: Plain-English Kid Metaphors to Senior DBMS Architecture

■ CORE PROJECT PHILOSOPHY

Executive Vision: *In modern computing, databases are the memory banks of society. When records get corrupted or disconnected, systems collapse. Traditional applications force human administrators to manually inspect messy SQL tables or write dangerous ad-hoc scripts. This project pioneers a self-healing database architecture that safely diagnoses broken records, calculates cascading blast radiuses using directed graphs, simulates fixes with non-destructive previews, and heals records within genuine ACID rollback-guaranteed transactions.*

Executive Summary & Academic Abstract

Relational Database Management Systems (RDBMS) serve as the foundation of modern data architecture, relying upon declarative constraints, candidate keys, and foreign key hierarchies to guarantee data consistency. However, in production enterprise environments, bulk ETL ingestion, partial batch jobs, accidental direct administrator edits, and unconstrained staging tables frequently introduce orphan records, candidate key collisions, mandatory attribute omissions, and domain range breaches. Blindly correcting or deleting these records introduces catastrophic cascading damage across dependent child tables.

This project presents an **Autonomous Self-Healing Relational Database Integrity System** built for MySQL InnoDB. The system decouples integrity management into a six-phase architectural lifecycle: dynamic metadata introspection via INFORMATION_SCHEMA, relational dependency modeling as a Directed Acyclic Graph (DAG) using NetworkX, deterministic multi-factor risk assessment, non-destructive before-and-after preview generation with cryptographic SHA-256 row-version concurrency tokens, atomic ACID transactional remediation with savepoints, and immutable audit event tracking enforced by database triggers.

The system completely avoids black-box machine learning models in favor of 100% deterministic, mathematically provable relational algebra queries, ensuring strict regulatory compliance, absolute repeatability, and zero phantom modifications.

Key Architectural Feature	Technical Implementation Mechanism	System Benefit Guarantee
Dynamic Introspection	Live INFORMATION_SCHEMA query evaluation	Zero hardcoded tables; adapts automatically to schema changes.
Dependency Modeling	NetworkX Directed Graph (DiGraph)	Identifies cycles, topological orders, and cascading blast radiuses.
Concurrency Defense	SHA-256 Canonical JSON Row Tokens	Rejects stale previews and prevents multi-user race condition overwrites.
Safe Remediation	Quarantine Landing Zone & Admin Input Guards	Zero data loss; invalid rows archived as JSON before source deletion.
Transaction Safety	InnoDB Autocommit=0, SAVEPOINT, ROLLBACK	All-or-nothing execution; automatic rollback on any operational failure.
Audit Immutability	Native BEFORE UPDATE & BEFORE DELETE triggers	Audit log entries physically impossible to alter or purge.

Table of Contents

Chapter	Chapter Title & Topical Coverage	Target Page
Chapter 1	The Intuitive Foundation: The 10-Year-Old Guide to Relational Databases	Page 4
Chapter 2	Academic Problem Statement, Background & Research Motivation	Page 6
Chapter 3	End-to-End System Architecture & Three-Tier Engineering Design	Page 8
Chapter 4	Complete Relational Database Specification (All 16 Tables Detailed)	Page 10
Chapter 5	Dynamic Schema Metadata Discovery via INFORMATION_SCHEMA	Page 20
Chapter 6	Relational Graph Theory & NetworkX Dependency Modeling	Page 22
Chapter 7	Cascading Blast Radius Analysis & Deterministic Risk Matrix	Page 23
Chapter 8	The Five Anomaly Taxonomies & Relational Algebra Detection Queries	Page 24
Chapter 9	The Autonomous Self-Healing Engine & Non-Destructive Previews	Page 27
Chapter 10	Cryptographic Row-Version Tokens & Concurrency Control (SHA-256)	Page 28
Chapter 11	Transaction Safeguards & ACID Property Implementation	Page 29
Chapter 12	Application-Level Recovery Engine & Snapshot Checkpoints	Page 31
Chapter 13	Audit Trail Immutability & Sensitive Credential Redaction Triggers	Page 32
Chapter 14	Comprehensive Walkthrough of All 11 Frontend Pages & Component Logic	Page 33
Chapter 15	Complete REST API Specification & Endpoint Contracts	Page 45
Chapter 16	Automated Testing Suite & Verification (All 45 Test Scenarios)	Page 49
Chapter 17	Local Deployment, Setup Wizard & Administration Guide	Page 52
Chapter 18	Oral Exam & Viva Defense Master Preparation (30 In-Depth Q&As;)	Page 54
Chapter 19	Technical Limitations, Security Hardening & Future Scope	Page 59
Chapter 20	Appendix, Project Glossary & File Manifest	Page 60

Chapter 1: The Intuitive Foundation: The 10-Year-Old Guide to Relational Databases

■ 10-YEAR-OLD INTUITION — THE INTUITIVE FOUNDATION: THE 10-YEAR-OLD GUIDE TO RELATIONAL DATABASES

Imagine a giant school made of Lego bricks with labeled drawers for Departments, Teachers, Classes, and Students. When clumsy hands write report cards for imaginary ghost kids or accidentally clone a student, our superhero robot doctor uses X-ray glasses to spot the mistakes, calculates if pulling one block will topple the entire tower, and uses a magical undo button if anything goes wrong!

1.1 The Giant Lego School: What is a Relational Database?

Computers store enormous amounts of information every second. Whether you are ordering a toy online, checking your school report card, or booking an airplane flight, your information lives inside a computer filing cabinet called a Relational Database Management System (RDBMS).

To understand what 'relational' means, imagine a school where everything is sorted into wooden drawers. One drawer holds a card for every Department (like Science or Art). Another drawer holds Teacher badges. A third drawer holds Classroom information, and a fourth drawer holds Student registration cards.

The word 'Relational' means that every drawer is connected to other drawers with invisible threads: a teacher badge only makes sense if it points to a real department, and a student's grade only makes sense if it points to both a real student and a real class. In computer science, these invisible threads are called Foreign Key Constraints.

- Table (Drawer): A labeled collection of cards sharing the same structure.
- Record or Row (Card): A single item, such as Student #101: Ravi Kumar.
- Primary Key (Unique ID Badge): A number that guarantees no two cards get mixed up.
- Foreign Key (Thread): A pointer from one card to another card's Primary Key.

1.2 Meet the Five Troublesome Data Monsters

Whenever humans type into computers or when systems import thousands of records from spreadsheets, mistakes happen. In our database school, five troublesome 'Data Monsters' sneak into the filing cabinets:

Data Monster	Everyday School Example	Technical Database Term	Danger Level
The Ghost Kid	A report card is written for Student #999, but nobody named #999 exists!	Orphan Record (Broken Foreign Key)	CRITICAL
The Evil Clone	Sneha is registered for Math 101 twice at the exact same hour.	Duplicate Candidate Key Collision	HIGH
The Invisible Student	A new student profile is filed, but the name is left completely empty.	Mandatory NULL / Empty String Breach	CRITICAL
The Time Traveler	A student's age is typed as 140 years old, or test score is 145/100.	Domain Range Violation	HIGH

The Secret Code	Enrollment status is written as 'SUPER-MAGIC' instead of 'ENROLLED'.	Domain Enum / Format Violation	MEDIUM

1.3 The Clumsy Robot Dilemma (The Domino Effect)

Why can't we just write a simple program that automatically deletes every mistake it finds? Because of the dreaded Domino Effect (Cascading Blast Radius).

Imagine a teacher's record contains a small spelling mistake. If a clumsy robot decides to solve the problem by deleting the teacher, what happens to the courses that teacher teaches? What happens to the grades of 100 students enrolled in those courses? If the database deletes everything connected to that teacher, half the school's records vanish!

This is why real enterprise databases cannot use blind automated deletion. Every fix must be carefully inspected, simulated with safety goggles, and reviewed by a human.

1.4 The Superhero Database Doctor: How Our System Solves It

Our project builds an Autonomous Self-Healing Relational Database System. It acts like a brilliant, careful robot doctor equipped with five superpowers:

- Superpower 1 (X-Ray Vision): Scans through thousands of records using SQL rules to locate violations without disturbing live operations.
- Superpower 2 (Family Tree Map): Uses graph algorithms (NetworkX) to map out who depends on whom, predicting the blast radius before touching anything.
- Superpower 3 (Hologram Preview): Projects a side-by-side preview showing the 'Before State' and the proposed 'After State' for human approval.
- Superpower 4 (Magical Undo Button): Steps inside an atomic ACID transaction bubble. If anything goes wrong, it hits ROLLBACK, and the database rewinds instantly.
- Superpower 5 (The Unbreakable Black Box): Writes down every single inspection, approval, and repair in an immutable audit diary protected by database triggers.

Chapter 2: Academic Problem Statement, Background & Research Motivation

■ 10-YEAR-OLD INTUITION — ACADEMIC PROBLEM STATEMENT, BACKGROUND & RESEARCH MOTIVATION

Big companies like banks and hospitals have millions of rows of data. When they copy data between old and new computers, things get disconnected. Normal websites don't check for these subtle errors. We built a system that actively hunts down these hidden bugs using mathematical logic instead of risky guesses.

2.1 The Reality of Enterprise Relational Data Degradation

In textbook relational database theory, declarative constraints (Primary Keys, Foreign Keys, UNIQUE indices, and CHECK clauses) are assumed to maintain perpetual integrity. In actual enterprise practice, however, data degradation is rampant.

Organizations routinely perform bulk ETL (Extract, Transform, Load) pipelines, staging migrations, external batch imports, and emergency direct administrator updates. To maximize throughput during ingestion, physical constraints are frequently disabled or omitted on staging tables, allowing corrupted, orphan, or duplicate records to enter relational storage.

Left uncorrected, these anomalies cascade into reporting errors, broken business logic, application crashes, and compromised analytical integrity.

2.2 Why Traditional CRUD Applications Fail at Database Integrity

Standard CRUD (Create, Read, Update, Delete) applications treat the database as a passive dumb store. They execute point queries against predefined form fields. When data integrity breaks, CRUD applications simply display error screens or silently return NULL.

In contrast, this project implements a Meta-Diagnostic Engine. It operates above the business application layer, continuously introspecting the schema itself, evaluating multi-table referential graphs, proposing safe healing actions, and verifying relational invariants.

Evaluation Dimension	Standard CRUD Application	Self-Healing DBMS Project
Primary Operational Focus	User record entry and retrieval	Autonomous anomaly diagnosis & healing
Schema Awareness	Hardcoded forms tied to static tables	Dynamic introspection of INFORMATION_SCHEMA
Relational Intelligence	Assumes all foreign keys are valid	Directed graph DAG modeling & blast radius
Repair Safety	Uncontrolled blind updates/deletes	Non-destructive previews & risk matrix
Transaction Rigor	Implicit single-query auto-commit	ACID multi-step isolation with savepoints
Audit Traceability	Simple access logs or none	Trigger-protected immutable event ledger

2.3 The Determinism Imperative: Why Machine Learning is Not Used

A critical engineering decision in this system is the deliberate rejection of machine learning or probabilistic heuristic models for database integrity enforcement.

In relational database engineering, data integrity is governed by mathematical formalisms (relational calculus and first-order predicate logic). An anomaly is not a statistical probability; it is a binary fact: a record either violates a referential constraint or it does not.

Deploying a black-box neural network to guess foreign-key mappings or infer surrogate keys risks hallucinating corrupted data. This project guarantees 100% deterministic, auditable, and mathematically provable outcomes through SQL predicate evaluation and graph traversal.

Chapter 3: End-to-End System Architecture & Three-Tier Engineering Design

> ■ **10-YEAR-OLD INTUITION — END-TO-END SYSTEM ARCHITECTURE & THREE-TIER ENGINEERING DESIGN**
>
> *Our system is built like a three-story building: the top floor is the colorful React dashboard where you click buttons, the middle floor is the smart Python FastAPI brain that does the math, and the ground floor is the super-secure MySQL vault where the data is locked safely.*

3.1 Architectural Decomposition

The Self-Healing Relational Database Integrity System adheres to a strict decoupled Three-Tier Architecture, ensuring high cohesion, modular testability, and zero cross-layer contamination:

1. Presentation Tier (Frontend): A responsive single-page application built on React 18, Vite 5, Tailwind CSS, and Recharts. Communicates exclusively via asynchronous JSON REST APIs.

2. Application Logic Tier (Backend): A high-performance Python 3.13 FastAPI service orchestrating specialized domain services: IntegrityScanner, DependencyAnalyzer, RepairEngine, TransactionManager, RecoveryManager, and AuditService.

3. Data Persistence Tier (Database): A MySQL 8.0+ / MariaDB 10.4+ engine configured with the InnoDB storage engine, utilizing row-level locking, foreign key cascades, and trigger-enforced immutability.

3.2 Directional Communication Flow & Proxy Routing

During local development and production deployment, the frontend application runs on port 5173, while the backend API runs on port 8000. All client API queries are routed through a reverse proxy configured in Vite (`/api -> http://127.0.0.1:8000`), completely eliminating Cross-Origin Resource Sharing (CORS) friction while maintaining strict browser isolation.

Component Module	Source Technology	Key Responsibility	Network Port
Dashboard UI	React 18 + Vite 5	Visualizations, interactive graph, modals	5173 (Browser)
REST Gateway	FastAPI + Uvicorn	Route handling, validation, JSON schemas	8000 (Internal)
Integrity Scanner	Python + PyMySQL	Dynamic metadata scanning & rule evaluation	Internal Service
Dependency Analyzer	NetworkX (DiGraph)	DAG traversal, cycle detection, blast radius	Internal Service
Repair Engine	Python + InnoDB SQL	Previews, SHA-256 tokens, ACID repair execution	Internal Service
Database Engine	MySQL InnoDB	ACID transactions, constraints, triggers	3306 (TCP)

3.3 InnoDB Concurrency & Lock Escalation Architecture

To guarantee that our self-healing repairs do not introduce concurrency deadlocks while operating in multi-threaded environments, the system relies on InnoDB's sophisticated lock escalation hierarchy. When a repair issues `SELECT ... FOR UPDATE`, InnoDB establishes Next-Key Locks across the index range, preventing phantom records from materializing during repair synthesis.

Lock Primitive	InnoDB Lock Type	Scope / Resource Protected	Concurrency Impact
Record Lock	LOCK_REC	Locks the exact physical index record.	Other transactions can modify adjacent records.
Gap Lock	LOCK_GAP	Locks the gap before an index record.	Prevents phantom insertions into ranges.
Next-Key Lock	LOCK_X | LOCK_GAP	Combines record lock and gap lock.	Default in REPEATABLE READ to eliminate phantom rows.
Table Intention	LOCK_IX	Intention-exclusive lock on table.	Enables fast table-level DDL conflict checks.

Chapter 4: Complete Relational Database Specification (All 16 Tables Detailed)

■ 10-YEAR-OLD INTUITION — COMPLETE RELATIONAL DATABASE SPECIFICATION (ALL 16 TABLES DETAILED)

Our database has 16 special rooms. Five rooms are for the school (teachers, students, classes), one is an unconstrained waiting room where new raw data lands, one is an emergency quarantine room for infected records, and the rest are control rooms and secret diaries that record every move.

4.1 Schema Categorization

The database instance `self_healing_test_db` consists of 16 tables divided into four functional tiers:

• Tier 1: Business Domain Schema (5 tables): `departments`, `instructors`, `courses`, `students`, `enrollments`.

• Tier 2: Staging & Quarantine Schema (2 tables): `staging_enrollments`, `quarantine_records`.

• Tier 3: Integrity Engine & Repair Schema (5 tables): `integrity_rules`, `scan_runs`, `integrity_issues`, `repair_recommendations`, `repair_executions`.

• Tier 4: Audit, Recovery & System Schema (4 tables): `audit_logs`, `transaction_logs`, `recovery_checkpoints`, `database_metadata`.

4.2 Business Domain Tables Specification

The business domain models an Academic Management System enforcing realistic relational hierarchies, check constraints, and cascading updates:

Table Name	Primary Key	Foreign Keys	Check Constraints & Invariants
departments	dept_id (INT)	None	CHECK (budget >= 0), dept_code UNIQUE
instructors	instructor_id (INT)	dept_id -> departments(dept_id)	CHECK (salary >= 0), email UNIQUE
courses	course_id (INT)	dept_id, instructor_id	CHECK (credits > 0 AND credits <= 12)
students	student_id (INT)	dept_id -> departments(dept_id)	email VARCHAR(150), age, gpa (3,2)
enrollments	enrollment_id (INT)	student_id, course_id	UNIQUE(student_id, course_id), grade_marks <= 100

4.3 The Role of Unconstrained Staging & Quarantine Tables

A cornerstone of the system's design is the architectural distinction between production tables (`enrollments`) and staging tables (`staging_enrollments`).

The staging table deliberately omits physical foreign key constraints. This simulates a real enterprise staging zone where external data imports (such as legacy system transfers or CSV batch uploads) can land without crashing the database, allowing our Integrity Scanner to inspect, isolate, and heal records prior to final production insertion.

When an irreparable or high-risk violation is detected, the `quarantine_records` table acts as a safe landing zone. The entire row is serialized into JSON with diagnostic reason tags before being safely excised from active tables, ensuring zero data loss.

4.4 System Engine, Audit & Recovery Tables Dictionary

The system maintains state, audit trails, and snapshot recovery points across nine dedicated engine tables:

Engine Table	Primary Key	Core Column Attributes	Engine Responsibility
integrity_rules	rule_id	rule_code, rule_type, target_table, parameters_json	Configurable catalog of validation predicates
scan_runs	scan_id	scan_type, start_time, end_time, issues_found, status	Execution history of all full/table inspection scans
integrity_issues	issue_id	scan_id, rule_code, table_name, record_id, status	Granular catalog of detected data anomalies
repair_recommendations	recommendation_id	issue_id, strategy, proposed_action, risk_level	Generated healing proposals awaiting approval
repair_executions	execution_id	issue_id, transaction_id, before_state, after_state	Permanent record of atomic repair executions
audit_logs	event_id	event_type, actor, timestamp, action_description	Immutable audit trail with credential redaction
transaction_logs	transaction_id	status, operations_count, rollback_reason, duration	Low-level transaction lifecycle recording
recovery_checkpoints	checkpoint_id	target_table, snapshot_rows_json, status	Pre-repair table state snapshots for rollback
database_metadata	meta_key	meta_value, updated_at	System key-value configuration and version store

4.5 Table Profile: `departments`

Technical specification and column schema dictionary for the `departments` table (Academic department organizational unit):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
dept_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary identifier for department.
dept_code	VARCHAR(20) NOT NULL	UNIQUE	Human-readable unique code (e.g., 'CS', 'EE').
dept_name	VARCHAR(100) NOT NULL	UNIQUE	Full official academic department name.
building	VARCHAR(100) NULL	None	Physical campus building location.
budget	DECIMAL(12,2) NOT NULL DEFAULT 100000.00	CHECK (budget >= 0)	Operating academic budget (must be non-negative).
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Record inception timestamp.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Record last modification timestamp.

4.6 Table Profile: `instructors`

Technical specification and column schema dictionary for the `instructors` table (Faculty instructor personnel entity):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
instructor_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary identifier for instructor.
instructor_name	VARCHAR(100) NOT NULL	None	Full legal name of faculty instructor.
email	VARCHAR(150) NOT NULL	UNIQUE	Institutional contact email address.
dept_id	INT NULL	FK -> departments(dept_id)	Department assignment (SET NULL on delete).
salary	DECIMAL(10,2) NOT NULL DEFAULT 50000.00	CHECK (salary >= 0)	Annual salary (must be non-negative).
hire_date	DATE NULL	None	Official date of academic appointment.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Record creation timestamp.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Record update timestamp.

4.7 Table Profile: `courses`

Technical specification and column schema dictionary for the `courses` table (Curricular course catalog offering):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
course_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary identifier for course.
course_code	VARCHAR(20) NOT NULL	UNIQUE	Catalog course code (e.g., 'CS101', 'EE101').
course_name	VARCHAR(150) NOT NULL	None	Full descriptive title of academic course.
credits	INT NOT NULL DEFAULT 3	CHECK (credits > 0 AND credits <= 12)	Academic credit weighting (1 to 12).
dept_id	INT NULL	FK -> departments(dept_id)	Offering academic department.
instructor_id	INT NULL	FK -> instructors(instructor_id)	Assigned primary course instructor.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Course registration timestamp.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Course catalog revision timestamp.

4.8 Table Profile: `students`

Technical specification and column schema dictionary for the `students` table (Enrolled student scholar entity):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
student_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary identifier for student.
student_name	VARCHAR(100) NOT NULL	None	Full student legal name.
email	VARCHAR(150) NOT NULL	None (Logical Candidate)	Student email address.
age	INT NULL	None (Domain Checked)	Student age in years (Domain bound 16-65).
gpa	DECIMAL(3,2) NULL	None (Domain Checked)	Cumulative grade point average (0.00-4.00).
dept_id	INT NULL	FK -> departments(dept_id)	Major academic department.
enrollment_year	INT NULL	None	Calendar year of matriculation.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Matriculation record timestamp.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Profile update timestamp.

4.9 Table Profile: `enrollments`

Technical specification and column schema dictionary for the `enrollments` table (Official student course enrollment junction):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
enrollment_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary key for enrollment.
student_id	INT NOT NULL	FK -> students(student_id)	Enrolled student reference (RESTRICT on delete).
course_id	INT NOT NULL	FK -> courses(course_id)	Enrolled course reference (RESTRICT on delete).
enrolled_on	DATE NOT NULL	None	Date registration was confirmed.
grade_marks	DECIMAL(5,2) NULL	CHECK (NULL or 0.00-100.00)	Exam grade marks percentage.
status	ENUM('ENROLLED','COMPLETED','DROPPED')	DEFAULT 'ENROLLED'	Academic lifecycle registration status.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Registration timestamp.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Grade modification timestamp.

4.10 Table Profile: `staging_enrollments`

Technical specification and column schema dictionary for the `staging_enrollments` table (Unconstrained raw staging data import sandbox):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
staging_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Staging record unique sequence identifier.
external_ref	VARCHAR(50) NULL	None	External legacy batch tracking identifier.

student_id	INT NULL	None (Unconstrained)	External student reference (monitored by scanner).
course_id	INT NULL	None (Unconstrained)	External course reference (monitored by scanner).
student_email	VARCHAR(150) NULL	None	Imported contact email.
enrolled_on	DATE NULL	None	Imported registration date.
grade_marks	DECIMAL(5,2) NULL	None (Unconstrained)	Imported raw score.
status	VARCHAR(50) NULL	None	Raw imported status string.
notes	VARCHAR(255) NULL	None	Staging diagnostic notes.
imported_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Staging ingestion timestamp.

4.11 Table Profile: `quarantine_records`

Technical specification and column schema dictionary for the `quarantine_records` table (Safe isolation landing zone for corrupted rows):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
quarantine_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate primary key for quarantine entry.
source_table	VARCHAR(100) NOT NULL	INDEX	Originating table from which row was excised.
source_pk_value	VARCHAR(100) NOT NULL	INDEX	Original primary key of quarantined record.
issue_type	VARCHAR(50) NOT NULL	None	Categorical violation tag (ORPHAN, DUPLICATE).
row_data_json	LONGTEXT NOT NULL	None	Full JSON serialization of complete original row.
quarantine_reason	TEXT NOT NULL	None	Human-readable explanation of why row was quarantined.
quarantined_by	VARCHAR(100) DEFAULT 'SYSTEM'	None	Operator or service identity performing quarantine.
quarantined_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Exact isolation timestamp.
resolved	BOOLEAN DEFAULT FALSE	None	Flag indicating whether record has been reviewed.
resolution_notes	TEXT NULL	None	Administrator remediation notes.

4.12 Table Profile: `integrity_rules`

Technical specification and column schema dictionary for the `integrity_rules` table (Configurable database integrity rulebook):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
rule_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate rule identifier.
rule_code	VARCHAR(50) NOT NULL	UNIQUE	Standardized rule code (e.g. RULE_ORPHAN_STUDENT).

name	VARCHAR(150) NOT NULL	None	Descriptive human-readable rule title.
description	TEXT NOT NULL	None	Detailed explanation of rule invariant.
rule_type	ENUM('ORPHAN','DUPLICATE','NULL_CHECK','DOMAIN_CHECK','RELATIONAL_CONSISTENCY')	INDEX	Rule anomaly classification.
target_table	VARCHAR(100) NOT NULL	INDEX	Table inspected by this rule.
target_column	VARCHAR(100) NOT NULL	None	Primary column evaluated.
parent_table	VARCHAR(100) NULL	None	Referenced parent table for orphan rules.
parent_column	VARCHAR(100) NULL	None	Referenced parent column for orphan rules.
parameters_json	LONGTEXT NULL	None	Dynamic parameters (min, max, enum, regex).
severity	ENUM('LOW','MEDIUM','HIGH','CRITICAL')	DEFAULT 'MEDIUM'	Risk severity weighting.
is_enabled	BOOLEAN NOT NULL DEFAULT TRUE	None	Runtime active toggle flag.

4.13 Table Profile: `scan_runs`

Technical specification and column schema dictionary for the `scan_runs` table (Full and table scan execution historical record):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
scan_id	VARCHAR(50) NOT NULL	PRIMARY KEY	Unique scan identifier (SCAN-YYYYMMDD-HEX).
scan_type	ENUM('FULL','TABLE') NOT NULL	DEFAULT 'FULL'	Scope of scan execution.
target_tables	TEXT NULL	None	Comma-separated target tables if TABLE scan.
start_time	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	INDEX	Scan commencement timestamp.
end_time	TIMESTAMP NULL	None	Scan completion timestamp.
status	ENUM('RUNNING','COMPLETED','FAILED')	INDEX	Lifecycle execution status.
total_rows_inspected	INT DEFAULT 0	None	Total live rows evaluated across tables.
issues_found	INT DEFAULT 0	None	Count of deduplicated violations discovered.
critical_count	INT DEFAULT 0	None	Count of CRITICAL severity violations.
high_count	INT DEFAULT 0	None	Count of HIGH severity violations.
medium_count	INT DEFAULT 0	None	Count of MEDIUM severity violations.
low_count	INT DEFAULT 0	None	Count of LOW severity violations.
error_message	TEXT NULL	None	Diagnostic error trace if scan failed.

4.14 Table Profile: `integrity_issues`

Technical specification and column schema dictionary for the `integrity_issues` table (Granular catalog of detected schema anomalies):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
issue_id	VARCHAR(50) NOT NULL	PRIMARY KEY	Unique issue identifier (ISS-YYYYMMDD-HEX).
scan_id	VARCHAR(50) NOT NULL	INDEX	Originating scan run identifier.
rule_id	INT NULL	None	Referenced integrity_rules row.
rule_code	VARCHAR(50) NOT NULL	None	Rule code that detected violation.
category	ENUM('ORPHAN','DUPLICATE','NULL_VIOLATION','DOMAIN_VIOLATION','RELATIONAL_VIOLATION','SYSTEM_INTEGRITY')	None	Violation taxonomy.
severity	ENUM('LOW','MEDIUM','HIGH','CRITICAL')	None	Severity level assigned.
table_name	VARCHAR(100) NOT NULL	INDEX	Table containing the defective row.
record_id	VARCHAR(100) NOT NULL	None	Primary key value of defective row.
column_name	VARCHAR(100) NULL	None	Specific column breaching integrity.
invalid_value	TEXT NULL	None	Current invalid value in database.
description	TEXT NOT NULL	None	Human-readable diagnostic description.
status	ENUM('OPEN','REVIEWED','APPROVED','REPAIRED','IGNORED','FAILED')	INDEX	Governance lifecycle state.
ignored_reason	TEXT NULL	None	Administrator rationale if marked IGNORED.
detected_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Detection timestamp.
repaired_at	TIMESTAMP NULL	None	Atomic repair completion timestamp.

4.15 Table Profile: `repair_recommendations`

Technical specification and column schema dictionary for the `repair_recommendations` table (Synthesized remediation action proposals):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
recommendation_id	INT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate proposal identifier.
issue_id	VARCHAR(50) NOT NULL	INDEX	Referenced integrity_issues row.
strategy	ENUM('RESTORE_PARENT','MAP_PARENT','QUARANTINE','DELETE_RECORD','MERGE_DUPLICATE','SET_DEFAULT','MANUAL_INTERVENTION')	None	Proposed healing action.

problem_description	TEXT NOT NULL	None	Summary of condition requiring repair.
proposed_action	TEXT NOT NULL	None	Specific healing remediation proposal.
reason	TEXT NOT NULL	None	Technical rationale for proposed strategy.
expected_impact	TEXT NOT NULL	None	Downstream schema impact summary.
risk_level	ENUM('LOW','MEDIUM','HIGH','CRITICAL')	None	Calculated risk classification.
is_reversible	BOOLEAN NOT NULL DEFAULT TRUE	None	Reversibility indicator flag.
requires_approval	BOOLEAN NOT NULL DEFAULT TRUE	None	Human approval requirement.
approval_status	ENUM('PENDING','APPROVED','REJECTED')	INDEX	Governance approval state.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Recommendation generation timestamp.

4.16 Table Profile: `repair_executions`

Technical specification and column schema dictionary for the `repair_executions` table (Permanent ledger of executed healing transactions):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
execution_id	VARCHAR(50) NOT NULL	PRIMARY KEY	Unique execution trace ID.
recommendation_id	INT NULL	None	Referenced recommendation ID.
issue_id	VARCHAR(50) NOT NULL	INDEX	Referenced issue ID.
transaction_id	VARCHAR(50) NOT NULL	None	Referenced InnoDB transaction ID.
status	ENUM('SUCCESS','FAILED','ROLLED_BACK')	INDEX	Final transaction outcome.
executed_by	VARCHAR(100) NOT NULL DEFAULT 'ADMIN_USER'	None	Operator identity.
executed_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Execution timestamp.
before_state_json	LONGTEXT NULL	None	Pre-repair row state JSON.
after_state_json	LONGTEXT NULL	None	Post-repair row state JSON.
sql_executed	TEXT NULL	None	Actual SQL DML statement executed.
error_message	TEXT NULL	None	Error details if rolled back.
execution_time_ms	INT DEFAULT 0	None	Execution duration in milliseconds.

4.17 Table Profile: `audit_logs`

Technical specification and column schema dictionary for the `audit_logs` table (Immutable event and state change ledger):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
event_id	BIGINT AUTO_INCREMENT NOT NULL	PRIMARY KEY	Surrogate audit sequence.
event_type	VARCHAR(50) NOT NULL	INDEX	Event type tag (SCAN_COMPLETED, REPAIR_EXECUTED).
actor	VARCHAR(100) NOT NULL DEFAULT 'SYSTEM'	None	Originating user or system actor.
timestamp	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	INDEX	Immutable event timestamp.
related_scan_id	VARCHAR(50) NULL	None	Linked scan run ID.
related_issue_id	VARCHAR(50) NULL	None	Linked issue ID.
related_transaction_id	VARCHAR(50) NULL	None	Linked transaction ID.
action_description	TEXT NOT NULL	None	Narrative description of action.
target_table	VARCHAR(100) NULL	INDEX	Table affected by event.
record_identifier	VARCHAR(100) NULL	None	Primary key affected.
result	ENUM('SUCCESS','FAILURE','INFO','WARNING')	None	Outcome indicator.
before_values_json	LONGTEXT NULL	None	Pre-event state with redaction.
after_values_json	LONGTEXT NULL	None	Post-event state with redaction.
error_message	TEXT NULL	None	Failure message if applicable.

4.18 Table Profile: `transaction_logs`

Technical specification and column schema dictionary for the `transaction_logs` table (Low-level InnoDB transaction lifecycle audit):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
transaction_id	VARCHAR(50) NOT NULL	PRIMARY KEY	Surrogate transaction trace ID.
execution_id	VARCHAR(50) NULL	None	Linked repair execution ID.
start_time	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	INDEX	Transaction begin timestamp.
end_time	TIMESTAMP NULL	None	Transaction completion timestamp.
status	ENUM('COMMITTED','ROLLED_BACK','FAILED')	INDEX	Transaction resolution status.
operations_count	INT DEFAULT 1	None	Number of distinct SQL statements.
savepoints_used	INT DEFAULT 0	None	Count of SAVEPOINT bookmarks set.
rollback_reason	TEXT NULL	None	Reason for abort if rolled back.
duration_ms	INT DEFAULT 0	None	Transaction duration in milliseconds.

4.19 Table Profile: `recovery_checkpoints`

Technical specification and column schema dictionary for the `recovery_checkpoints` table (Table state snapshots for point-in-time rollback):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
checkpoint_id	VARCHAR(50) NOT NULL	PRIMARY KEY	Surrogate checkpoint identifier (CKPT-XXXXXX).
checkpoint_name	VARCHAR(150) NOT NULL	None	Descriptive checkpoint name.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	None	Snapshot creation timestamp.
checkpoint_type	ENUM('PRE_REPAIR','MANUAL','SCHEDULED')	None	Trigger context of checkpoint.
target_table	VARCHAR(100) NOT NULL	None	Table captured by snapshot.
snapshot_metadata_json	LONGTEXT NOT NULL	None	Table DDL and column schema JSON.
snapshot_rows_json	LONGTEXT NOT NULL	None	Complete row dataset JSON.
status	ENUM('ACTIVE','RESTORED','EXPIRED')	INDEX	Checkpoint availability status.

4.20 Table Profile: `database_metadata`

Technical specification and column schema dictionary for the `database_metadata` table (System key-value persistent configuration store):

Column Name	Type & Nullability	Key / Constraint	Functional Purpose
meta_key	VARCHAR(100) NOT NULL	PRIMARY KEY	Unique configuration key name.
meta_value	TEXT NOT NULL	None	Stored configuration or timestamp value.
updated_at	TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	None	Last modification timestamp.

Chapter 5: Dynamic Schema Metadata Discovery via INFORMATION_SCHEMA

■ 10-YEAR-OLD INTUITION — DYNAMIC SCHEMA METADATA DISCOVERY VIA INFORMATION_SCHEMA

Instead of hardcoding table names like a lazy robot, our system walks into the database library and reads the master card catalog (called INFORMATION_SCHEMA). It discovers all columns, keys, and rules dynamically at runtime!

5.1 Why Hardcoded Schemas Are Prohibited

A major flaw of legacy DBMS academic projects is hardcoding table names and column definitions directly into backend controllers. If a new column is added or a table is renamed, hardcoded systems immediately crash.

The Self-Healing System utilizes dynamic metadata reflection. When initialized, the engine queries MySQL's internal `INFORMATION_SCHEMA` tables to inspect table types, column nullability, primary key constraints, foreign key mappings, and index topologies on the fly.

5.2 SQL Injection Defense via Strict Identifier Allowlisting

Because table and column names in SQL cannot be bound using standard parameter substitution (`?` or `%s`), dynamic schema exploration poses an inherent SQL Injection risk if user input is concatenated into queries.

To guarantee total security, our database layer implements the `validate_identifier()` guardian. Every table and column identifier must pass a strict regex allowlist before being used in any dynamic query:

```
IDENTIFIER_REGEX = re.compile(r'^[a-zA-Z_][a-zA-Z0-9_]*$')

def validate_identifier(name: str) -> str:
    if not name or not IDENTIFIER_REGEX.match(name):
        raise ValueError(f'Security Violation: Invalid SQL identifier: {name}')
    return name
```

5.3 Core Introspection Queries

The backend executes standard introspection queries against MySQL catalogs:

```
-- 1. Discover all active user tables
SELECT table_name, table_type, engine, table_rows
FROM information_schema.tables
WHERE table_schema = 'self_healing_test_db';

-- 2. Discover detailed column definitions and constraints
SELECT column_name, data_type, is_nullable, column_default, column_key
FROM information_schema.columns
WHERE table_schema = 'self_healing_test_db' AND table_name = %s
ORDER BY ordinal_position;

-- 3. Discover all physical foreign key relationships
SELECT table_name AS child_table, column_name AS child_column,
       referenced_table_name AS parent_table, referenced_column_name AS parent_column
FROM information_schema.key_column_usage
WHERE table_schema = 'self_healing_test_db' AND referenced_table_name IS NOT NULL;
```

5.4 Query Optimization & MySQL EXPLAIN Plan Inspection

To guarantee that detection queries do not cause locking contention or table degradation on large datasets, every query was profiled using MySQL's `EXPLAIN` optimizer. Below is the optimizer execution profile for the core anti-join:

Select Type	Table Scanned	Access Type	Possible Keys	Key Used	Rows Examined	Filter Rationale
SIMPLE	c (staging_enrollments)	ALL	None	None	8	Full staging scan for external batch inspection.
SIMPLE	p (students)	eq_ref	PRIMARY	PRIMARY	1	Index primary key lookup per child row.
SIMPLE	departments	ALL	None	None	4	Full table metadata reflection during scan bootstrap.
SIMPLE	integrity_issues	ref	idx_issue_status	idx_issue_status	11	Indexed status search for OPEN triage issues.

Chapter 6: Relational Graph Theory & NetworkX Dependency Modeling

■ 10-YEAR-OLD INTUITION — RELATIONAL GRAPH THEORY & NETWORKX DEPENDENCY MODELING

We turn all 16 tables into a giant Family Tree using dots and arrows (a Directed Graph). An arrow points from parents to children. We use smart math to check if the arrows form a trap loop (cycles) and calculate the exact safe order to clean up rooms without tripping over each other!

6.1 Mathematical Formulation of the Relational Graph

Relational integrity enforcement requires understanding the multi-tier dependency topology between tables. The system models the database as a Directed Graph using the `NetworkX` library:

$$G = (V, E)$$

Where vertices V represent database tables, and directed edges $e = (P, C) \in E$ represent referential dependencies pointing from the parent (referenced) table P to the child (referencing) table C.

6.2 Cycle Detection and Prevention Algorithm

In relational design, circular foreign keys (Table A references Table B, which references Table C, which references Table A) create catastrophic locking deadlocks during cascading operations. The `DependencyAnalyzer` actively evaluates acyclicity:

$$\text{IsAcyclic} \iff \text{nx.is_directed_acyclic_graph}(G) = \text{True}$$

If cycles exist, the engine executes `nx.simple_cycles(G)` to locate every circular loop, alerts administrators on the dashboard, and disables automatic cascading deletes to avoid infinite recursion.

6.3 Topological Sorting for Safe Multi-Table Operations

When performing data imports or cleanups, operations must occur in strict dependency order. In reverse topological order, child tables are cleared or detached before parent tables are touched:

$$\text{DeletionOrder} = \text{list}(\text{reversed}(\text{list}(\text{nx.topological_sort}(G))))$$

This guarantees that referential integrity constraints are never violated during bulk migrations.

Topological Rank	Table Node	Relationship Type	Dependency Rationale
Rank 1 (Root Parents)	departments	Independent Root	Has zero incoming foreign keys; safe to seed first.
Rank 2 (Direct Dependents)	instructors, students	Level 1 Dependents	Depend directly on departments; require valid dept_id.
Rank 3 (Catalog Nodes)	courses	Level 2 Dependents	Depends on both departments and instructors.
Rank 4 (Terminal Junctions)	enrollments	Leaf Child Junction	Depends on both students and courses; must be seeded last.

Chapter 7: Cascading Blast Radius Analysis & Deterministic Risk Matrix

■ 10-YEAR-OLD INTUITION — CASCADING BLAST RADIUS ANALYSIS & DETERMINISTIC RISK MATRIX

Before we touch any broken record, we calculate its 'Blast Radius'—just like checking how many standing dominoes will fall if you push the first one over! If touching one record will hurt 50 other records, the system screams: 'CRITICAL RISK! DANGER!' and requires extra human confirmation.

7.1 Mathematical Definition of Relational Blast Radius

The Relational Blast Radius defines the maximum theoretical boundary of data that could be corrupted, altered, or orphaned if a modification or deletion is applied to table T:

$$\text{BlastRadius}(T) = \text{Descendants}(G, T) = \{ u \in V \mid \exists \text{ directed path from } T \text{ to } u \}$$

$$\text{BlastRadiusTableCount}(T) = |\text{Descendants}(G, T)|$$

For example, deleting a record in `departments` has a downstream blast radius spanning `instructors`, `courses`, `students`, and `enrollments` ($|\text{BlastRadius}| = 4$).

7.2 Live Referential Row Introspection

Knowing which tables *could* be affected is not enough. The `DependencyAnalyzer` inspects actual live rows in the database to count exactly how many child rows currently point to the specific target primary key:

$$\text{MatchingChildRows}(r_{\text{pk}}) = \sum_{C \in \text{Children}(P)} \text{COUNT}(*) \text{ WHERE } C.\text{fk} = r_{\text{pk}}$$

7.3 The Multi-Factor Risk Decision Matrix

The system classifies repair proposals into risk tiers using a deterministic multi-factor decision matrix:

Issue Severity	Repair Action	Live Dependent Rows?	Downstream Children?	Assigned Risk Tier
CRITICAL	DELETE / QUARANTINE	Yes (> 0)	Yes	CRITICAL (Maximum Danger)
CRITICAL	SET_DEFAULT	No (0)	No	HIGH
HIGH	DELETE / QUARANTINE	Any	Yes	CRITICAL
HIGH	MAP_PARENT	No (0)	No	HIGH
MEDIUM	DELETE / QUARANTINE	Yes (> 0)	Yes	CRITICAL
MEDIUM	SET_DEFAULT	No (0)	Yes	MEDIUM
LOW	SET_DEFAULT	No (0)	No	LOW (Safe Automated)

Chapter 8: The Five Anomaly Taxonomies & Relational Algebra Detection Queries

> ■ **10-YEAR-OLD INTUITION — THE FIVE ANOMALY TAXONOMIES & RELATIONAL ALGEBRA DETECTION QUERIES**
>
> *We wrote special magnifying-glass queries for all five types of mistakes: 1) LEFT JOIN searches for ghost kids, 2) GROUP BY searches for twins, 3) IS NULL searches for invisible names, 4) Inequality signs search for 140-year-old students, and 5) Regex searches for emails missing the @ sign!*

8.1 Anomaly Category 1: Orphan Records (Broken Foreign Keys)

An orphan record is a child row containing a non-NULL foreign key pointing to a primary key value that does not exist in the referenced parent table.

$$\mathcal{O} = \{ r \in \text{Child} \mid r.\text{fk} \neq \text{NULL} \land \neg\exists p \in \text{Parent} (p.\text{pk} = r.\text{fk}) \}$$

```
-- Detection SQL using relational anti-join
SELECT c.staging_id, c.student_id, c.external_ref
FROM staging_enrollments c
LEFT JOIN students p ON c.student_id = p.student_id
WHERE p.student_id IS NULL AND c.student_id IS NOT NULL;
```

8.2 Anomaly Category 2: Duplicate Candidate Keys (Entity Collision)

A candidate key duplicate occurs when multiple records share identical logical attributes that should uniquely identify an entity, bypassing physical constraints in staging tables.

$$\mathcal{D} = \{ g \in \text{Groups}(K) \mid \text{COUNT}(g) > 1 \}$$

```
-- Detection SQL grouping candidate keys
SELECT student_id, course_id, COUNT(*) AS group_count,
       GROUP_CONCAT(staging_id ORDER BY staging_id ASC) AS all_ids
FROM staging_enrollments
GROUP BY student_id, course_id
HAVING COUNT(*) > 1;
```

8.3 Anomaly Category 3: Mandatory NULL and Empty String Breaches

Occurs when an essential attribute lacks a valid value, including both physical database NULLs and whitespace-padded empty strings.

$$\text{Violation}(r) \iff (r.\text{col} \text{ IS NULL}) \lor (\text{TRIM}(r.\text{col}) = '')$$

```
-- Detection SQL for mandatory attribute omission
SELECT student_id, student_name
FROM students
WHERE student_name IS NULL OR TRIM(student_name) = '';
```

8.4 Anomaly Category 4: Domain Range Violations

Occurs when numeric values violate real-world operational bounds, such as test marks outside $[0.00, 100.00]$ or student age outside $[16, 65]$.

$$\text{Violation}(x) \iff (x < \text{MinVal}) \lor (x > \text{MaxVal})$$

```
-- Detection SQL for domain range violations
SELECT staging_id, grade_marks
FROM staging_enrollments
WHERE grade_marks < 0.00 OR grade_marks > 100.00;
```

8.5 Anomaly Category 5: Domain Enumeration & Regex Syntax Violations

Enforces finite set membership for state variables and structural compliance for formatted strings:

$$\text{EnumViolation}(s) \iff s \notin \{ \text{'ENROLLED'}, \text{'COMPLETED'}, \text{'DROPPED'} \}$$

$$\text{RegexViolation}(e) \iff e \not\sim \text{`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`}$$

```
-- Detection SQL for status codes and email syntax
SELECT staging_id, status FROM staging_enrollments
WHERE status NOT IN ('ENROLLED', 'COMPLETED', 'DROPPED');
```

8.6 The Complete Catalog of 45 Seeded Integrity Rules Across All Six Tables

The system initializes with 45 active, parameterized integrity rules spanning all four operational categories: ORPHAN referential checks (7 rules), DUPLICATE candidate key checks (7 rules), NULL completeness checks (14 rules), and DOMAIN constraint checks (17 rules).

The rule coverage spans all six database tables: departments (5 rules), instructors (6 rules), courses (7 rules), students (8 rules), enrollments (9 rules), and staging_enrollments (10 rules).

Below is the complete 45-rule integrity configuration matrix detailing target entities, monitored attributes, severity classifications, and autonomous self-healing strategies:

Rule Code	Category	Target Table	Target Column	Severity	Strategy
RULE_DUP_DEPT_CODE	DUPLICATE	departments	dept_code	CRITICAL	DELETE / MERGE
RULE_DUP_DEPT_NAME	DUPLICATE	departments	dept_name	HIGH	DELETE / MERGE
RULE_NULL_DEPT_CODE	NULL_CHECK	departments	dept_code	CRITICAL	SET_DEFAULT
RULE_NULL_DEPT_NAME	NULL_CHECK	departments	dept_name	CRITICAL	SET_DEFAULT
RULE_DOMAIN_DEPT_BUDGET	DOMAIN_CHECK	departments	budget	HIGH	SET_DEFAULT
RULE_ORPHAN_INSTRUCTOR_DEPT	ORPHAN	instructors	dept_id	HIGH	MAP_PARENT
RULE_DUP_INSTRUCTOR_EMAIL	DUPLICATE	instructors	email	CRITICAL	DELETE / MERGE

RULE_NULL_INSTRUCTOR_NAME	NULL_CHECK	instructors	instructor_name	CRITICAL	SET_DEFAULT
RULE_NULL_INSTRUCTOR_EMAIL	NULL_CHECK	instructors	email	CRITICAL	SET_DEFAULT
RULE_DOMAIN_INSTRUCTOR_SALARY	DOMAIN_CHECK	instructors	salary	HIGH	SET_DEFAULT
RULE_DOMAIN_INSTRUCTOR_EMAIL	DOMAIN_CHECK	instructors	email	MEDIUM	SET_DEFAULT
RULE_ORPHAN_COURSE_DEPT	ORPHAN	courses	dept_id	HIGH	MAP_PARENT
RULE_ORPHAN_COURSE_INSTRUCTOR	ORPHAN	courses	instructor_id	HIGH	MAP_PARENT
RULE_DUP_COURSE_CODE	DUPLICATE	courses	course_code	CRITICAL	DELETE / MERGE
RULE_NULL_COURSE_CODE	NULL_CHECK	courses	course_code	CRITICAL	SET_DEFAULT
RULE_NULL_COURSE_NAME	NULL_CHECK	courses	course_name	HIGH	SET_DEFAULT
RULE_NULL_COURSE_CREDITS	NULL_CHECK	courses	credits	HIGH	SET_DEFAULT
RULE_DOMAIN_COURSE_CREDITS	DOMAIN_CHECK	courses	credits	HIGH	SET_DEFAULT
RULE_ORPHAN_STUDENT_DEPT	ORPHAN	students	dept_id	HIGH	MAP_PARENT
RULE_DUP_STUDENT_EMAIL	DUPLICATE	students	email	CRITICAL	DELETE / MERGE
RULE_NULL_STUDENT_NAME	NULL_CHECK	students	student_name	CRITICAL	SET_DEFAULT
RULE_NULL_STUDENT_EMAIL	NULL_CHECK	students	email	CRITICAL	SET_DEFAULT
RULE_DOMAIN_STUDENT_AGE	DOMAIN_CHECK	students	age	HIGH	SET_DEFAULT
RULE_DOMAIN_STUDENT_GPA	DOMAIN_CHECK	students	gpa	HIGH	SET_DEFAULT
RULE_DOMAIN_EMAIL_FORMAT	DOMAIN_CHECK	students	email	MEDIUM	SET_DEFAULT
RULE_DOMAIN_STUDENT_ENROLL_YEAR	DOMAIN_CHECK	students	enrollment_year	MEDIUM	SET_DEFAULT
RULE_ORPHAN_ENROLLMENT_STUDENT	ORPHAN	enrollments	student_id	CRITICAL	MAP_PARENT
RULE_ORPHAN_ENROLLMENT_COURSE	ORPHAN	enrollments	course_id	CRITICAL	MAP_PARENT
RULE_DUP_ENROLLMENT_PAIR	DUPLICATE	enrollments	student_id,course_id	CRITICAL	DELETE / MERGE
RULE_NULL_ENROLLMENT_STUDENT	NULL_CHECK	enrollments	student_id	CRITICAL	SET_DEFAULT
RULE_NULL_ENROLLMENT_COURSE	NULL_CHECK	enrollments	course_id	CRITICAL	SET_DEFAULT

RULE_NULL_ENROLLMENT_STATUS	NULL_CHECK	enrollments	status	HIGH	SET_DEFAULT
RULE_NULL_ENROLLMENT_DATE	NULL_CHECK	enrollments	enrolled_on	HIGH	SET_DEFAULT
RULE_DOMAIN_ENROLLMENT_GRADE	DOMAIN_CHECK	enrollments	grade_marks	HIGH	SET_DEFAULT
RULE_DOMAIN_ENROLLMENT_STATUS_ENUM	DOMAIN_CHECK	enrollments	status	HIGH	SET_DEFAULT
RULE_ORPHAN_STAGING_STUDENT	ORPHAN	staging_enrollments	student_id	CRITICAL	QUARANTINE
RULE_ORPHAN_STAGING_COURSE	ORPHAN	staging_enrollments	course_id	CRITICAL	QUARANTINE
RULE_DUP_STAGING_ENROLLMENT	DUPLICATE	staging_enrollments	student_id,course_id	HIGH	DELETE / MERGE
RULE_DUP_STAGING_EXT_REF	DUPLICATE	staging_enrollments	external_ref	MEDIUM	DELETE / MERGE
RULE_NULL_STAGING_STUDENT	NULL_CHECK	staging_enrollments	student_id	HIGH	SET_DEFAULT
RULE_NULL_STAGING_COURSE	NULL_CHECK	staging_enrollments	course_id	HIGH	SET_DEFAULT
RULE_NULL_STAGING_STATUS	NULL_CHECK	staging_enrollments	status	HIGH	SET_DEFAULT
RULE_DOMAIN_MARKS_RANGE	DOMAIN_CHECK	staging_enrollments	grade_marks	HIGH	SET_DEFAULT
RULE_DOMAIN_ENROLLMENT_STATUS	DOMAIN_CHECK	staging_enrollments	status	MEDIUM	SET_DEFAULT
RULE_DOMAIN_STAGING_EMAIL	DOMAIN_CHECK	staging_enrollments	student_email	MEDIUM	SET_DEFAULT

Chapter 9: The Autonomous Self-Healing Engine & Non-Destructive Previews

■ 10-YEAR-OLD INTUITION — THE AUTONOMOUS SELF-HEALING ENGINE & NON-DESTRUCTIVE PREVIEWS

Before any surgery, our doctor puts on safety goggles and shows the human operator a side-by-side hologram: 'Here is the sick record on the left, here is how it will look healed on the right, and here is the exact SQL command.' We have 4 healing strategies: Quarantine, Default Values, Parent Re-linking, and Safe Deletion.

9.1 The Five-Stage Healing Lifecycle

To prevent catastrophic administrative accidents, every self-healing operation passes through five mandatory governance stages:

1. Detection: The scanner identifies the anomaly and inserts an `OPEN` issue.

2. Synthesis: The engine generates a structured `repair_recommendations` entry.

3. Preview Generation: The engine constructs a non-mutating preview capturing the Before State, simulated After State, SQL query, and risk rating.

4. Operator Approval: High-risk and critical repairs require explicit operator confirmation; values for empty fields must be provided by the administrator.

5. Transactional Execution: The engine executes the repair inside an atomic InnoDB transaction protected by row locks and recovery checkpoints.

9.2 The Four Supported Healing Strategies

The system implements four distinct healing strategies designed for different anomaly contexts:

Strategy Code	Operational Action	Data Loss Risk	Typical Usage Context
QUARANTINE	Archives entire row as JSON in quarantine_records, then deletes from active table.	Zero Data Loss (Archived)	Corrupted staging rows with invalid parent references.
SET_DEFAULT	Updates the defective column with a validated default or admin-provided value.	Zero Data Loss (In-Place)	Missing names, empty emails, or out-of-range scores.
MAP_PARENT	Updates the broken foreign key to point to an authoritative existing parent ID.	Zero Data Loss (Re-linked)	Orphan staging enrollments with known parent students.
DELETE_RECORD	Permanently deletes the duplicate or unrecoverable record from the source table.	Irreversible Deletion	Identical duplicate submissions in staging tables.

Chapter 10: Cryptographic Row-Version Tokens & Concurrency Control

■ 10-YEAR-OLD INTUITION — CRYPTOGRAPHIC ROW-VERSION TOKENS & CONCURRENCY CONTROL

What if two people try to fix the same record at the exact same second? Our system takes a digital fingerprint (SHA-256 hash) of the record during the preview. If anyone sneaks in and changes even one letter before you click Confirm, the fingerprint won't match, and the system shouts: 'STOP! The record has changed!'

10.1 The Phantom Modification Problem

In multi-user database systems, a severe vulnerability exists known as a Stale Preview Hazard. An administrator opens a repair preview for Record #8. While they are reviewing the screen, another process or user edits or deletes Record #8. If the administrator clicks 'Execute', the repair would blindly overwrite the newly modified data.

To achieve absolute concurrency control, the `RepairEngine` implements an Optimistic Concurrency Token based on a SHA-256 cryptographic digest.

10.2 Canonical JSON Hashing Algorithm

The row state is normalized into a sorted, deterministic dictionary where all keys are sorted alphabetically and all datetime objects are converted to ISO 8601 strings:

$$\text{CanonicalRow} = \text{SortKeys}(\{ k: \text{Normalize}(v) \mid (k, v) \in \text{Row} \})$$

$$\text{RowVersionToken} = \text{SHA-256}(\text{UTF8}(\text{JSON}(\text{CanonicalRow})))$$

```
# Exact Token Verification at Execution Time
current_token = hashlib.sha256(
    json.dumps(current_row, sort_keys=True, default=str).encode('utf-8')
).hexdigest()

if expected_token and current_token != expected_token:
    raise StalePreviewError(
        'Concurrency Conflict: The record was modified by another process '
        'after this preview was generated. Please refresh the preview.'
    )
```

10.3 Pessimistic Row Locking (`SELECT ... FOR UPDATE`)

Once the token matches, the engine switches to Pessimistic Locking during execution. It issues `SELECT * FROM tbl WHERE id = %s FOR UPDATE`. This tells MySQL InnoDB to acquire an exclusive row-level write lock, preventing any other concurrent transaction from reading or altering the row until our healing transaction finishes.

Chapter 11: Transaction Safeguards & ACID Property Implementation

■ 10-YEAR-OLD INTUITION — TRANSACTION SAFEGUARDS & ACID PROPERTY IMPLEMENTATION

ACID is a superhero shield for databases! It means: Atomicity (All or Nothing), Consistency (Never break the rules), Isolation (Don't let others peek while cooking), and Durability (Once written, it stays forever even if power cuts off). We built interactive simulations demonstrating all of them!

11.1 The InnoDB ACID Engine in Practice

Relational integrity cannot exist without strict transaction management. The project relies on the MySQL InnoDB engine with `autocommit=False` to enforce the four ACID pillars:

• Atomicity: An entire repair operation executes as a single unit. If any sub-query fails, `conn.rollback()` restores the exact pre-transaction state.

• Consistency: Triggers and foreign keys validate relational invariants before committing.

• Isolation: REPEATABLE READ transaction isolation ensures queries see a consistent snapshot.

• Durability: Once `conn.commit()` is confirmed, write-ahead redo logs guarantee persistence.

11.2 The Three Interactive Demonstration Scenarios

The application includes an interactive Transaction Simulator executing real InnoDB operations on the live server:

Scenario Name	Operational Sequence	Expected State Verification	ACID Principle Proven
COMMIT_SUCCESS	START TRANSACTION -> Insert record -> Verify intermediate state -> COMMIT	Final Count = Initial Count + 1	Durability & Atomic State Transition
SIMULATED_FAILURE_ROLLBACK	START TRANSACTION -> Insert record -> Encounter simulated error -> ROLLBACK	Final Count = Initial Count (Zero rows leaked)	Atomicity (All-or-Nothing Rollback)
SAVEPOINT_PARTIAL_ROLLBACK	START TRANSACTION -> Insert A -> SAVEPOINT sp1 -> Insert B -> ROLLBACK TO sp1 -> COMMIT	Record A persists; Record B completely revoked	Selective Transaction Recovery (Savepoints)

11.3 Transaction SQL Traces

The exact SQL sequence executed during partial savepoint rollback:

```
START TRANSACTION; /* Autocommit disabled */
INSERT INTO staging_enrollments (staging_id, external_ref, student_id, course_id, status)
VALUES (9001, 'TX-A', 101, 1, 'ENROLLED');
SAVEPOINT sp_step1;
INSERT INTO staging_enrollments (staging_id, external_ref, student_id, course_id, status)
VALUES (9002, 'TX-B', 999, 999, 'INVALID');
ROLLBACK TO SAVEPOINT sp_step1;
COMMIT;
-- Verification: Record 9001 exists; Record 9002 was undone!
```

11.4 Granular Step-by-Step Transaction Execution Matrix

Below is the exact execution trace and state transition matrix recorded by `TransactionManager` during a `SAVEPOINT_PARTIAL_ROLLBACK` operational run:

Step #	Operation Executed	SQL Statement Dispatched	Intermediate DB State	Isolation Level
Step 1	BEGIN TRANSACTION	START TRANSACTION /* Autocommit=0 */	Snapshot isolation view established	REPEATABLE READ
Step 2	INSERT RECORD A	INSERT INTO staging_enrollments VALUES (9001, 'TX-A', ...)	Dirty row visible ONLY inside connection	Uncommitted
Step 3	SET SAVEPOINT	SAVEPOINT step_a_complete	Named transaction recovery bookmark saved	Savepoint Active
Step 4	INSERT RECORD B	INSERT INTO staging_enrollments VALUES (9002, 'TX-B', ...)	Temporary invalid row staged	Uncommitted
Step 5	SIMULATED FAILURE	Runtime Exception Triggered (Constraint / Network Error)	Engine traps error and halts execution	Error Trapped
Step 6	PARTIAL ROLLBACK	ROLLBACK TO SAVEPOINT step_a_complete	Record B undone; Record A state preserved!	Partial Revert
Step 7	COMMIT TRANSACTION	COMMIT /* Redo log written to disk */	Record A permanently persisted to InnoDB tables	DURABLE

Chapter 12: Application-Level Recovery Engine & Snapshot Checkpoints

■ 10-YEAR-OLD INTUITION — APPLICATION-LEVEL RECOVERY ENGINE & SNAPSHOT CHECKPOINTS

Before the doctor makes any changes, it takes a high-resolution photocopy of the whole room. If the human operator later changes their mind, they click 'Restore Checkpoint', and the room magically reverts to the exact snapshot photocopy!

12.1 Why Table-Level Checkpoints Are Necessary

Transaction rollback only protects operations within the active database connection. Once a transaction is committed, standard database rollbacks cannot undo it.

To provide post-commit recovery, the `RecoveryManager` captures application-level snapshots. Before executing any high-risk repair, the engine captures a pre-repair checkpoint recording table DDL and row states as structured JSON inside `recovery_checkpoints`.

12.2 Checkpoint Capturing and Restoration Workflow

During snapshot creation:

1. Captures `SHOW CREATE TABLE tbl` to store table structure.

2. Captures `SELECT * FROM tbl` to serialize all rows into JSON.

3. Assigns a unique ID (`CKPT-XXXXXX`) and marks status as `ACTIVE`.

During restoration, the engine verifies that snapshot row counts match actual rows, checks foreign key constraints, upserts missing records, and deletes records added after the checkpoint was captured.

12.3 Mathematical State Verification

A restoration is only committed if post-restore verification satisfies two invariants:

$$\text{ExpectedRowCount} = |\text{SnapshotRows}|$$

$$\text{ActualRowCount} = \text{SELECT COUNT}(*) \text{ FROM TargetTable}$$

$$\text{Verified} \iff (\text{ExpectedRowCount} = \text{ActualRowCount}) \land (\forall r \in \text{SnapshotRows}, r \in \text{TargetTable})$$

Chapter 13: Audit Trail Immutability & Sensitive Credential Redaction

■ 10-YEAR-OLD INTUITION — AUDIT TRAIL IMMUTABILITY & SENSITIVE CREDENTIAL REDACTION

We have an unbreakable black box like on an airplane! Every scan, repair, and transaction is logged. If anyone tries to change or erase the log, MySQL triggers block them. And if someone types a secret password, the system automatically writes [REDACTED]!

13.1 Regulatory Audit Compliance in DBMS

Enterprise data compliance frameworks (such as SOX, HIPAA, and GDPR) mandate that database alterations must maintain a verifiable, tamper-evident audit trail.

The `AuditService` records all lifecycle events into the `audit_logs` table, storing actor identification, event type, affected record primary keys, before-and-after JSON snapshots, execution timing, and outcome status.

13.2 Database Triggers Blocking UPDATE and DELETE

Application-level logging is insufficient if an administrator can directly execute `DELETE FROM audit_logs`. To achieve physical immutability, the schema installs native MySQL triggers that abort any update or deletion attempt with SQLSTATE 45000:

```
CREATE TRIGGER `trg_audit_logs_no_update`
BEFORE UPDATE ON `audit_logs`
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'CRITICAL: audit_logs entries are immutable and cannot be updated.';
END;

CREATE TRIGGER `trg_audit_logs_no_delete`
BEFORE DELETE ON `audit_logs`
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'CRITICAL: audit_logs entries are immutable and cannot be deleted.';
END;
```

13.3 Sensitive Credential Redaction Filter

Before serializing before-and-after states into audit JSON, all dictionaries pass through the redaction sanitizer. Any key matching sensitive substrings (`password`, `secret`, `token`, `pwd`, `hash`, `credit_card`) is masked with `[REDACTED]` to prevent credential leakage in audit records.

Chapter 14: Comprehensive Walkthrough of All 11 Frontend Pages

> ■ **10-YEAR-OLD INTUITION — COMPREHENSIVE WALKTHROUGH OF ALL 11 FRONTEND PAGES**
>
> *The website has 11 awesome screens: 1) Master Monitor, 2) Glass Cabinet Explorer, 3) Family Tree Visualizer, 4) X-Ray Scanner, 5) Doctor's Problem List, 6) Surgery Preview Room, 7) Time Machine Sandbox, 8) Photocopy Recovery Room, 9) Unbreakable Diary, 10) School Rulebook, and 11) Testing Lab!*

14.1 Page 1: System Health Dashboard (`/`) — Comprehensive Component Architecture

The System Health Dashboard serves as the central flight control room of the application, rendering live database telemetry, anomaly distribution charts, and real-time operational event streams. Below is the granular breakdown of all five core components on this page:

14.1.1 Component 1: Live Server Health & Latency Telemetry Gauge

• Visual Wireframe Role: Header pill displaying a green pulsing status indicator, connected database name (`self_healing_test_db`), MySQL engine version, and round-trip TCP socket ping latency in milliseconds.

• 10-Year-Old Explanation: The Doctor's Stethoscope. Just like a stethoscope listens to your heartbeat, this widget constantly pings the database vault to check if MySQL is awake and how fast it answers.

• Underlying DBMS Concept: Client-Server Socket Communication and Telemetry Ping. Measures connection establishment overhead, network buffer round-trip time, and thread pool responsiveness in MySQL.

• Directional Data Flow: On component mount, the React `useEffect` hook dispatches an asynchronous Axios GET request to `/api/health`. FastAPI invokes `test_database_connection()` via PyMySQL, which records start time, executes metadata queries, records completion time, and computes latency delta.

• Mathematical Formula: Latency in milliseconds is calculated as: $$\text{Latency}_{\text{ms}} = (t_{\text{response}} - t_{\text{dispatch}}) \times 1000$$

```
-- Live Telemetry Diagnostic Query
SELECT VERSION() AS server_version, CURRENT_USER() AS connected_user;
-- Verifies that MySQL server process is alive and responsive.
```

14.1.2 Component 2: Overall Database Health Score StatCard & Radial Progress Meter

• Visual Wireframe Role: Prominent top card displaying an overall 0-100% database integrity purity score with dynamic color grading (Emerald Green > 90%, Amber 70-90%, Crimson Red < 70%) and a circular SVG progress track.

• 10-Year-Old Explanation: The School Health Report Card. Like getting an A+ grade on a school test when all your homework is turned in, this gauge tells you how close your database is to 100% perfection.

• Underlying DBMS Concept: Relational Integrity Ratio and Anomaly Prevalence Metric. Quantifies data consistency by measuring the proportion of clean records against unique records compromised by referential, domain, or candidate key violations.

• Directional Data Flow: React fetches open issue counts from `/api/issues?status=OPEN` and total table row counts from `/api/database/status`. The frontend memoizes the score using `useMemo` and dynamically animates the SVG circular stroke offset.

• Mathematical Formula: The Health Score formula balances total rows against unique compromised entities: $$\text{Health Score} = \max\left(0, \; 100 - \left(\frac{U_{\text{issues}}}{N_{\text{total_records}}} \times 100 \right)\right)$$

```
-- Query to calculate total schema records
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'self_healing_test_db';

-- Query to count unique open anomalous records
SELECT COUNT(DISTINCT table_name, record_id) AS unique_broken_rows
FROM integrity_issues
WHERE status = 'OPEN';
```

14.1.3 Component 3: Anomaly Severity Distribution Pie Chart

• Visual Wireframe Role: Interactive Recharts SVG Pie Chart breaking down open violations into four color-coded slices: CRITICAL (Red `#EF4444`), HIGH (Orange `#F97316`), MEDIUM (Amber `#F59E0B`), and LOW (Blue `#3B82F6`), complete with animated tooltips and percentage legends.

• 10-Year-Old Explanation: The Danger Pie. A colorful pizza chart showing how many big emergencies (broken bones) vs medium scrapes vs tiny papercuts exist in the school drawers.

• Underlying DBMS Concept: Priority Queue Triage and Constraint Severity Weighting. Organizes database violations by their potential for catastrophic cascading side-effects.

• Directional Data Flow: Backend groups `integrity_issues` by severity enum. The resulting JSON dictionary is mapped into Recharts `Pie` data array with custom `Cell` fills.

• Mathematical Formula: S-slice percentage: $$\text{Percentage}(s) = \left(\frac{\text{Count}(s)}{\sum_{k \in \{\text{CRITICAL, HIGH, MEDIUM, LOW}\}} \text{Count}(k)} \right) \times 100$$

```
-- Grouping open issues by severity tier
SELECT severity, COUNT(*) AS issue_count
FROM integrity_issues
WHERE status = 'OPEN'
GROUP BY severity
ORDER BY FIELD(severity, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW');
```

14.1.4 Component 4: Anomaly Category Distribution Bar Chart

• Visual Wireframe Role: Recharts Vertical Bar Chart displaying violation volume across the five fundamental anomaly taxonomies (ORPHAN, DUPLICATE, NULL_CHECK, DOMAIN_CHECK, RELATIONAL).

• 10-Year-Old Explanation: The Monster Comparison Towers. Tower blocks showing whether we have more Ghost Kids, more Evil Twins, or more Missing Names.

• Underlying DBMS Concept: Relational Algebra Anomaly Taxonomy. Distinguishes between referential foreign key anti-joins, candidate key collisions, domain boundary violations, and nullability breaches.

• Directional Data Flow: API endpoint `/api/database/status` issues a SQL `GROUP BY category` query. Frontend transforms keys into human-readable labels and binds to SVG ``.

• Mathematical Formula: Category Rank: $$\text{Rank}(C) = \text{ArgSort}(\text{Count}(C))$$

```
-- Grouping issues across categorical taxonomy
SELECT category, COUNT(*) AS count
FROM integrity_issues
GROUP BY category
ORDER BY count DESC;
```

14.1.5 Component 5: Recent Scan Runs & Live Audit Stream Feed

• Visual Wireframe Role: Dual split scrollable data tables rendering the last 5 completed scan executions (Scan ID, Type, Issues Detected, Timing) and the last 5 audit events with user actor badges.

• 10-Year-Old Explanation: The Doctor's Daily Diary. A running list showing who opened the drawers, who ran the x-ray machine, and what changes were made in the last hour.

• Underlying DBMS Concept: Temporal Event Streaming and Transactional History Audit. Provides immediate situational awareness and operational traceability.

• Directional Data Flow: Dispatched via parallel Axios `Promise.all([api.getScans({limit: 5}), api.getAuditLogs({limit: 5})])`. Renders responsive glassmorphic table rows.

```
-- Fetch recent scan runs
SELECT scan_id, scan_type, start_time, total_rows_inspected, issues_found, status
FROM scan_runs ORDER BY start_time DESC LIMIT 5;

-- Fetch recent audit events
SELECT event_id, event_type, actor, timestamp, action_description, result
FROM audit_logs ORDER BY timestamp DESC LIMIT 5;
```

14.2 Page 2: Database Explorer (`/database`) — Interactive Catalog Inspection

The Database Explorer allows administrators to inspect live schema structures, column data types, foreign key hierarchies, and paginated row data directly from MySQL catalogs.

14.2.1 Component 1: Searchable Table Selector Dropdown & Tier Badges

• Visual Wireframe Role: Searchable combobox dropdown listing all 16 tables categorized with colored tier pills: Blue for Business Domain, Purple for Staging, Emerald for Safety/Quarantine, and Slate for System Engine.

• 10-Year-Old Explanation: The Magic Drawer Key. You pick which drawer you want to open and inspect with your flashlight.

• Underlying DBMS Concept: Data Catalog Reflection via `INFORMATION_SCHEMA.TABLES`. Avoids hardcoding and reflects new tables automatically upon database schema revision.

• Directional Data Flow: When selected, updates React state `selectedTable`. Triggers parallel API requests to fetch schema columns (`/api/database/schema/{table}`) and live row data (`/api/database/tables/{table}/data`).

```
SELECT table_name, table_type, engine, table_rows
FROM information_schema.tables
WHERE table_schema = 'self_healing_test_db'
ORDER BY table_name ASC;
```

14.2.2 Component 2: Schema Metadata Specification Grid Tab

• Visual Wireframe Role: Interactive specification table displaying Column Name, MySQL Physical Storage Type, Nullable Constraint (`YES`/`NO`), Default Expression, Primary Key badge (Gold Key icon), and Foreign Key target references.

• 10-Year-Old Explanation: The Drawer's Instruction Manual. Shows what kind of Lego bricks fit into each slot (numbers, names, dates) and which slots are strictly required.

• Underlying DBMS Concept: Data Definition Language (DDL) Dictionary. Dynamic query against `INFORMATION_SCHEMA.COLUMNS` exposing column ordinal positions and storage attributes.

• Directional Data Flow: Renders column array into responsive styled table rows with copy-to-clipboard icons for quick column name reference.

```
SELECT column_name, data_type, column_type, is_nullable, column_default, column_key
FROM information_schema.columns
WHERE table_schema = 'self_healing_test_db' AND table_name = %s
ORDER BY ordinal_position;
```

14.2.3 Component 3: Live Data Grid with Offset Pagination & Sorting

• Visual Wireframe Role: Responsive data grid rendering live records with previous/next page controls, row limit selector (25, 50, 100), and clickable column sort headers.

• 10-Year-Old Explanation: The Glass Window. Lets you look straight into the drawer to see the actual student cards and teacher badges.

• Underlying DBMS Concept: Relational Selection (σ), Projection (π), and Offset-Based Buffer Pagination. Avoids memory exhaustion by fetching rows in bounded batches.

• Mathematical Formula: $$\text{Offset} = (\text{PageNumber} - 1) \times \text{PageLimit}$$

• Exact SQL: `SELECT * FROM \`courses\` ORDER BY \`course_id\` ASC LIMIT 50 OFFSET 0;`

14.2.4 Component 4: Dynamic SQL Injection Prevention Guardian

• Visual Wireframe Role: Security validation middleware intercepting any table or column parameter before SQL dispatch.

• 10-Year-Old Explanation: The Security Bouncer. Checks every name card to make sure no trickster wrote code to blow up the school.

• Underlying DBMS Concept: Identifier Allowlisting and Parameterized Isolation. Because table names in SQL cannot be bound with prepared statements, strict regex filtering is mandatory.

• Mathematical Formula: $$\text{IsValidIdentifier}(w) \iff w \in \mathcal{L}\left(\text{`^[a-zA-Z_][a-zA-Z0-9_]*$`}\right)$$

14.3 Page 3: Relational Dependency Visualizer (`/dependencies`) — DAG Topology

The Dependency Visualizer maps the complete relational schema as an interactive Directed Acyclic Graph (DAG), rendering parent-child foreign key hierarchies and computing cascading blast radiuses.

14.3.1 Component 1: Interactive SVG Directed Graph Canvas

• Visual Wireframe Role: Full-screen interactive vector canvas rendering database tables as circular nodes and foreign keys as directed bezier curves with glowing arrowheads.

• 10-Year-Old Explanation: The Big Family Tree Map. Circles represent drawers, and arrows show which drawer holds hands with which other drawer.

• Underlying DBMS Concept: Directed Graph $G = (V, E)$ modeling foreign key relationships where edge (P, C) represents child table C referencing parent table P.

14.3.2 Component 2: Schema Acyclicity & Cycle Detection Indicator

• Visual Wireframe Role: Header pill displaying a green checkmark 'Schema is Acyclic (DAG)' or a flashing red warning badge 'Circular Referential Loop Detected!'.

• 10-Year-Old Explanation: The Loop Warning Bell. Rings if the arrows form a trap circle (A needs B, B needs C, C needs A) that would jam the computer forever.

• Underlying DBMS Concept: Directed Cycle Detection and Deadlock Hazard Prevention using `NetworkX`.

• Mathematical Formula: $$\text{HasCycle}(G) \iff \text{nx.is_directed_acyclic_graph}(G) = \text{False}$$

14.3.3 Component 3: Table Neighborhood Inspector Sidebar

• Visual Wireframe Role: Split sidebar panel displaying the selected table's Direct Parents, Direct Children, Upstream Ancestors, and Downstream Descendants.

• 10-Year-Old Explanation: The Family Inspector. Click on 'courses' and it shows you its parents ('departments', 'instructors') and its children ('enrollments').

• Underlying DBMS Concept: Subgraph Extraction, In-Degree/Out-Degree Analysis, and Transitive Reachability.

• Mathematical Formulas: $$\text{Parents}(T) = \{ u \mid (u, T) \in E \}, \quad \text{Children}(T) = \{ v \mid (T, v) \in E \}$$

• Exact SQL: Queries `INFORMATION_SCHEMA.KEY_COLUMN_USAGE` to isolate referenced and referencing table pairs.

14.3.4 Component 4: Cascading Blast Radius Calculator

• Visual Wireframe Role: Interactive impact calculator allowing operators to enter a Record ID and compute exactly how many downstream child rows would be impacted by modification.

• 10-Year-Old Explanation: The Domino Counter. Calculates how many standing dominoes will topple over if you knock down this one card.

• Underlying DBMS Concept: Cascading Referential Integrity and Transitive Blast Radius Analysis.

• Mathematical Formula: $$\text{BlastRadius}(T) = \text{Descendants}(G, T), \quad |\text{BlastRadius}| = \text{TableCount}$$

• Exact SQL: Executes `SELECT COUNT(*) FROM child WHERE fk_col = %s` for each child table.

14.4 Page 4: Integrity Scanner (`/scanner`) — Six-Table Autonomous Anomaly Detection Console

The Integrity Scanner serves as the autonomous diagnostic core of the platform, executing 45 parameterized integrity rules across all six schema tables (departments, instructors, courses, students, enrollments, staging_enrollments) to identify referential orphans, candidate key duplicates, mandatory null breaches, and domain anomalies.

14.4.1 Component 1: Six-Table Target Scope Selector & Table Checkbox Matrix

• Visual Wireframe Role: Three-panel top configuration card providing Full DB (all six core tables) vs Selected scope toggles, individual table checkboxes with row counts, and dynamic table inclusion.

• 10-Year-Old Explanation: The Search Zone Chooser. Tells the robot doctor whether to inspect all six classrooms in the school or just look inside the Staging Sandbox.

• Underlying DBMS Concept: Scan Scope Partitioning and Workload Isolation. Allows running targeted inspections against single tables (e.g. `staging_enrollments`) without locking or scanning unrelated production entities.

• Monitored Tables: `departments` (Reference Entity), `instructors` (Faculty Entity), `courses` (Curricular Entity), `students` (Scholar Entity), `enrollments` (Junction Entity), `staging_enrollments` (Raw Intake Sandbox).

14.4.2 Component 2: 45 Active Rules Configuration & Category Breakdown Cards

• Visual Wireframe Role: Rule selector panel displaying all 45 enabled rules with severity badges, coupled with four post-scan Category Breakdown Cards: ORPHAN Checks, DUPLICATE Checks, NULL Checks, and DOMAIN Checks.

• 10-Year-Old Explanation: The Four Emergency Clocks. Four big cards showing how many Ghost Kids (orphans), Evil Twins (duplicates), Blank Nametags (nulls), and Impossible Scores (domain errors) were found.

• Underlying DBMS Concept: Formal Integrity Constraint Taxonomy (Codd & Date). Divides relational verification into Referential Integrity (foreign keys), Entity Integrity (candidate uniqueness), Completeness (non-nullness), and Domain Integrity (ranges, enums, regexes).

• Directional Data Flow: Scans API `/api/scans` returns aggregated `issues_by_category` dictionary, updating the interactive filter state upon click.

14.4.3 Component 3: Six-Table Integrity Coverage Matrix

• Visual Wireframe Role: Comprehensive summary table displaying each of the six schema tables, its relational role, total rules configured, total rows inspected, total issues detected, category count badges, and real-time health status badge.

• 10-Year-Old Explanation: The School Report Board. A big leaderboard showing every single classroom drawer: how many cards were read, how many were broken, and whether that drawer is completely clean or needs medicine.

• Underlying DBMS Concept: Schema-Wide Multi-Relation Health Indexing. Aggregates inspection metrics across parent entities, dependent child entities, junction relations, and unconstrained staging tables.

• Exact SQL Diagnostics: Aggregates row counts via `COUNT(*)` on each table and sums issues grouped by `table_name` from `integrity_issues`.

14.4.4 Component 4: Severity Distribution Bar & Multi-Table Findings Data Table

• Visual Wireframe Role: Multi-colored stacked distribution bar (CRITICAL Red, HIGH Orange, MEDIUM Yellow, LOW Blue) and interactive findings table with table pills (`All`, `departments`, `instructors`, etc.), category pills, severity pills, real-time search box, and direct 'Self-Heal' action buttons.

• 10-Year-Old Explanation: The Symptom Searcher. Lets you filter by classroom or type a student's name to instantly find their broken record and click the 'Heal' button.

• Directional Data Flow: Clicking 'Self-Heal' navigates the operator directly to `/repairs?issue=ISSUE-XXXXXX`, pre-loading the ACID transaction preview modal for instant remediation.

14.5 Page 5: Issue Management Center (`/issues`) — Triage & Governance

The Issue Management Center provides an administrative triage console for filtering, inspecting, reviewing, and approving detected database violations.

14.5.1 Component 1: Multi-Filter Pill Bar (Status & Severity)

• Visual Wireframe Role: Interactive filter pills allowing instant multi-criteria filtering by Lifecycle Status (`OPEN`, `REVIEWED`, `APPROVED`, `REPAIRED`, `IGNORED`, `FAILED`) and Severity.

• 10-Year-Old Explanation: The Sorting Bins. Click 'Emergency' to see only the red cards, or click 'Fixed' to see cards the doctor already healed.

• Underlying DBMS Concept: Finite State Machine (FSM) Lifecycle Governance. Enforces that issues transition through formal review states before healing.

• Mathematical State Transition: $$\text{OPEN} \xrightarrow{\text{review}} \text{REVIEWED} \xrightarrow{\text{approve}} \text{APPROVED} \xrightarrow{\text{execute}} \text{REPAIRED}$$

14.5.2 Component 2: Live Raw Record JSON Inspector Modal

• Visual Wireframe Role: Modal window displaying the live database record formatted as syntax-highlighted JSON, allowing operators to verify the surrounding row context.

• 10-Year-Old Explanation: The Record Magnifying Glass. Lets you read the entire student card to see their grade, email, and age all at once.

• Underlying DBMS Concept: Point Primary Key Fetch. Issues a parameterized `SELECT * FROM tbl WHERE pk = %s` against the live InnoDB table.

14.5.3 Component 3: Administrative Triage Actions (Review & Ignore)

• Visual Wireframe Role: Action buttons allowing operators to mark an issue as 'Reviewed' or mark as 'Ignored' with a mandatory modal prompting for administrator justification notes.

• 10-Year-Old Explanation: The Doctor's Notes. You can stamp the card as 'Checked' or write: 'Ignore this because it's a test student.'

• Exact SQL: `UPDATE integrity_issues SET status = 'IGNORED', ignored_reason = %s WHERE issue_id = %s;`

14.6 Page 6: Repair Center & Non-Destructive Preview Room (`/repairs`) — Multi-Table Safe Remediation

The Repair Center is the surgical theater of the system, enforcing non-destructive Before/After comparisons, optimistic concurrency tokens, multi-table filtering across all six tables, and ACID rollback protection.

14.6.1 Component 1: Multi-Table Healing Recommendation Selector & Filter Bar

• Visual Wireframe Role: Header filter pill bar supporting instant filtering across all six schema tables (`All (6)`, `departments`, `instructors`, `courses`, `students`, `enrollments`, `staging_enrollments`), repair strategy, and resolution status.

• 10-Year-Old Explanation: The Medicine Shelf. Lets you pick which drawer's broken toys you want to repair first, or choose only the big emergency operations.

• Underlying DBMS Concept: Entity-Scoped Remediation Queuing. Isolates pending repair recommendations by target relational table, ensuring operators can triage staging data or production tables independently.

14.6.2 Component 2: Before State vs Proposed After State Split Cards

• Visual Wireframe Role: Side-by-side split cards comparing the exact current row in the database against the proposed healed state, with modified fields highlighted in emerald green.

• 10-Year-Old Explanation: The Before-and-After Hologram. Shows what the card looks like right now on the left, and what it will look like when healed on the right.

• Underlying DBMS Concept: Non-Destructive In-Memory Simulation. Constructs the repaired row state in Python application memory without issuing write locks or modifying the live database.

14.6.3 Component 3: Cryptographic SHA-256 Row-Version Token Card

• Visual Wireframe Role: Card displaying the 64-character SHA-256 hash computed from the row state, guaranteeing that the record has not been altered since preview generation.

• 10-Year-Old Explanation: The Digital Fingerprint. If anyone sneaks in and changes the card while you are looking at the screen, the fingerprint changes and the doctor stops!

• Underlying DBMS Concept: Optimistic Concurrency Control (OCC) and Phantom Modification Defense.

• Mathematical Formula: $$\text{Token} = \text{SHA-256}\left(\text{JSON}(\text{SortKeys}(\text{Row})) \right)$$

14.6.4 Component 4: Quarantine Zero-Data-Loss JSON Archival Engine

• Visual Wireframe Role: Dedicated quarantine notice banner detailing the three-phase archival protocol: 1) Full row serialization to JSON; 2) Atomic insertion into `quarantine_records`; 3) Deletion from source table only after verified storage.

• 10-Year-Old Explanation: The Safe Storage Box. Before throwing away a messy paper, the computer takes a high-definition photograph of it and locks it in the museum safe, so nothing is ever lost forever.

• Underlying DBMS Concept: Transaction-Safe Isolation and Zero Data Loss (ZDL) Invariant. Guarantees that corrupted staging rows are preserved with full audit provenance.

• Exact SQL: `INSERT INTO quarantine_records (source_table, source_pk_value, issue_type, row_data_json, quarantine_reason) VALUES (%s, %s, %s, %s, %s);` followed by `DELETE FROM staging_enrollments WHERE staging_id = %s;` within an atomic transaction.

14.6.5 Component 5: Parameterized SQL Preview Box & Two-Stage Confirmation Dialog

• Visual Wireframe Role: Syntax-highlighted code block displaying the exact SQL statement (`UPDATE` or `INSERT ... DELETE`) that will be dispatched inside the transaction, followed by a modal confirmation dialog.

• 10-Year-Old Explanation: The Exact Spell and Double Lock. Shows the exact words whispered to MySQL and requires a two-button confirmation before committing changes.

14.7 Page 7: Transaction Simulator (`/transactions`) — ACID Laboratory

The Transaction Simulator provides an interactive educational laboratory demonstrating MySQL InnoDB transaction isolation, commit durability, automatic failure rollbacks, and partial savepoint recovery.

14.7.1 Component 1: Interactive Scenario Selector Cards

• Visual Wireframe Role: Three selectable scenario cards: 1) `COMMIT_SUCCESS` (Atomicity & Durability), 2) `SIMULATED_FAILURE_ROLLBACK` (All-or-Nothing Rollback), and 3) `SAVEPOINT_PARTIAL_ROLLBACK` (Selective Undo).

• 10-Year-Old Explanation: The Time Machine Game Mode. You pick whether you want to test saving the game, crashing and rewinding, or using a secret bookmark.

• Underlying DBMS Concept: InnoDB Transaction Isolation and Multi-Statement ACID Boundaries.

14.7.2 Component 2: Step-by-Step Vertical Stepper Timeline

• Visual Wireframe Role: Vertical timeline rendering each transaction milestone (`START TRANSACTION`, `INSERT`, `SAVEPOINT`, `ROLLBACK`, `COMMIT`) with millisecond duration timers.

• 10-Year-Old Explanation: The Story Steps. Shows every move the computer made second-by-second.

• Directional Data Flow: Dispatches `POST /api/transactions/demo` with scenario payload. Renders returned `steps` array into interactive stepper.

14.7.3 Component 3: State Snapshot & Atomicity Verification Badge

• Visual Wireframe Role: Banner comparing Initial Table Row Count against Final Table Row Count, displaying a green 'Atomicity Verified: Initial Count = Final Count' badge on rollback.

• 10-Year-Old Explanation: The Proof Badge. Proves that when the crash happened, zero fake cards were left behind.

• Mathematical Verification: $$\Delta N = N_{\text{final}} - N_{\text{initial}} = 0 \quad (\text{Atomic Rollback Confirmed})$$

14.8 Page 8: Recovery Center (`/recovery`) — Snapshot Management

The Recovery Center manages application-level table snapshots, providing one-click point-in-time state restoration with mathematical row verification.

14.8.1 Component 1: Create Table Checkpoint Form

• Visual Wireframe Role: Form allowing operators to select any table, assign a custom snapshot name, select snapshot type (`MANUAL`, `PRE_REPAIR`), and capture an instant checkpoint.

• 10-Year-Old Explanation: The Photocopy Button. Takes a magic snapshot picture of the entire drawer and saves it in a safe vault.

• Exact SQL: Executes `SHOW CREATE TABLE tbl` and `SELECT * FROM tbl` to serialize schema and data as JSON into `recovery_checkpoints`.

14.8.2 Component 2: Active Checkpoints History Grid

• Visual Wireframe Role: Data table displaying Checkpoint ID (`CKPT-XXXXXX`), Target Table, Creation Timestamp, Captured Row Count, and Status (`ACTIVE`, `RESTORED`).

• 10-Year-Old Explanation: The Snapshot Photo Album. Shows all the saved pictures you can restore back to.

• Underlying DBMS Concept: Point-in-Time Application Recovery and Snapshot State Cataloging.

14.8.3 Component 3: One-Click Restore & Verification Modal

• Visual Wireframe Role: Modal dialog that executes transactional restoration, deletes post-snapshot rows, upserts snapshot rows, and verifies post-state.

• 10-Year-Old Explanation: The Magic Restore Wand. Pops the drawer back to exactly how it looked in the snapshot picture.

• Mathematical Invariant: $$\text{RestorationVerified} \iff (\text{Count}(\text{Current}) = \text{Count}(\text{Snapshot})) \land (\text{PKs Match})$$

14.9 Page 9: Audit Trail Explorer (`/audit`) — Compliance Ledger

The Audit Trail Explorer exposes the immutable flight recorder of all system events, complete with credential redaction and trigger immutability notices.

14.9.1 Component 1: Audit Event Search & Filter Bar

• Visual Wireframe Role: Search input filtering by Table Name, Record Identifier, Actor Identity (`ADMIN_USER`, `SYSTEM`), and Event Type (`SCAN_COMPLETED`, `REPAIR_EXECUTED`).

• 10-Year-Old Explanation: The Secret Diary Searcher. Lets you search for any day or any student to see who touched their card.

• Exact SQL: `SELECT * FROM audit_logs WHERE target_table = %s ORDER BY timestamp DESC LIMIT 50;`

14.9.2 Component 2: Payload Inspector Modal with Redaction Tags

• Visual Wireframe Role: Modal window displaying Before Values JSON and After Values JSON, showing sensitive keys masked with `[REDACTED]` tags.

• 10-Year-Old Explanation: The Secret Marker. If someone wrote a password, the system covers it with black marker so nobody can steal it.

• Underlying DBMS Concept: Data Privacy and Regulatory Masking (HIPAA / GDPR / SOX compliance).

14.9.3 Component 3: Database Trigger Immutability Notice

• Visual Wireframe Role: Permanent banner explaining that audit entries cannot be altered or deleted due to MySQL `BEFORE UPDATE` and `BEFORE DELETE` triggers.

• 10-Year-Old Explanation: The Steel Lock. Explains that this diary is locked in steel; even the principal cannot erase pages.

• Exact SQL: `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Audit logs are immutable.';`

14.10 Page 10: Rules Configuration (`/rules`) — Policy Management

The Rules Configuration page allows administrators to inspect, toggle, edit, and dry-run test validation rules across the schema.

14.10.1 Component 1: Rule Cards Grid with Parameter Tags

• Visual Wireframe Role: Responsive card grid displaying Rule Code, Target Table, Column, Severity Badge, and JSON parameter badges (e.g. `min: 16, max: 65`).

• 10-Year-Old Explanation: The School Rulebook Posters. Cards on the wall showing the rules for each classroom.

• Underlying DBMS Concept: Declarative Metadata Rule Catalog stored in `integrity_rules`.

14.10.2 Component 2: Runtime Active / Inactive Toggle Switches

• Visual Wireframe Role: Instant toggle switches allowing operators to activate or deactivate any rule with immediate visual feedback.

• 10-Year-Old Explanation: The Light Switches. Turn a rule on or off like flicking a classroom light switch.

• Exact SQL: `UPDATE integrity_rules SET is_enabled = %s WHERE rule_id = %s;`

14.10.3 Component 3: Dry-Run Test Rule Modal

• Visual Wireframe Role: Modal that immediately executes the rule's SQL predicate against live tables and reports matching violation counts without modifying data.

• 10-Year-Old Explanation: The Practice Drill. Tests the rule right now to see how many kids break it before officially turning it on.

• Directional Data Flow: Dispatches `POST /api/rules/{id}/test`. Executes rule query via PyMySQL and returns detected count.

14.11 Page 11: Settings & Diagnostics (`/settings`) — Testing Laboratory

The Settings page provides system telemetry, environment configuration inspection, and demo data manipulation controls.

14.11.1 Component 1: Database Connection Telemetry Card

• Visual Wireframe Role: Card displaying Host (`127.0.0.1`), Port (`3306`), Database (`self_healing_test_db`), Character Set (`utf8mb4`), and round-trip ping time.

• 10-Year-Old Explanation: The Engine Room Speedometer. Shows the computer cables and how fast data is traveling.

• Underlying DBMS Concept: Database Diagnostic Handshake and Connection Pool Health.

14.11.2 Component 2: 'Inject Demo Violations' Trigger Button

• Visual Wireframe Role: Action button that executes `database/05_demo_violations.sql`, inserting 11 controlled anomalies across staging and student tables.

• 10-Year-Old Explanation: The Practice Mess Maker. Plants 11 pretend mistakes into the school drawers so you can demonstrate the robot doctor to your teacher!

• Exact SQL: Truncates `staging_enrollments` and inserts records with orphan student 999, orphan course 888, marks 145, age 140, GPA 4.95, and invalid status.

14.11.3 Component 3: 'Reset to Clean Baseline' Trigger Button

• Visual Wireframe Role: Action button that executes `database/07_reset_demo_data.sql`, restoring pristine baseline academic data and clearing temporary anomalies.

• 10-Year-Old Explanation: The Magic Eraser. Wipes away all the practice mistakes and makes the school 100% spotless and clean again.

• Exact SQL: Deletes demo students with `student_id >= 200`, resets staging enrollments, and logs a baseline reset audit event.

Chapter 15: Complete REST API Specification & Endpoint Contracts

■ 10-YEAR-OLD INTUITION — COMPLETE REST API SPECIFICATION & ENDPOINT CONTRACTS

The backend speaks a universal language called JSON. Every response comes inside a neat envelope with four stamps: success (did it work?), message (what happened?), data (the answer!), and error (what went wrong, if any).

15.1 Standardized RESTful Response Architecture & Envelope Pattern

To ensure strict predictability and type safety across all frontend components, the FastAPI backend wraps every HTTP response in a standardized JSON envelope.

Regardless of whether an endpoint delivers a scalar health metric, a graph topology payload, or a tabular schema dictionary, the payload structure adheres to the following formal contract:

1. `success` (Boolean): Master execution indicator denoting whether the relational operation completed without unhandled exceptions.

2. `data` (T | None): Generic typed payload containing the queried domain entity or mutation result. Evaluates to null on errors.

3. `message` (String): Human-readable operational message describing the outcome of the request.

4. `timestamp` (ISO-8601 String): High-precision UTC timestamp recording the moment of response generation on the backend server.

5. `error_code` (String | Optional): Standardized error identifier (e.g., `CONCURRENCY_TOKEN_MISMATCH`, `REFERENTIAL_CASCADE_BLOCK`) provided during non-200 responses.

15.2 The Comprehensive Catalog of All 24 REST Endpoints

The table below documents every REST API route exposed by the FastAPI server, detailing HTTP verbs, URIs, operational categories, and payload contracts:

Method	Endpoint URI	Category	Request Body / Params	Success Response Type
GET	/api/health	System	None	HealthStatusSchema (status, latency_ms, db)
GET	/api/database/status	Database	None	DbStatusSchema (tables, total_rows, engine)
GET	/api/database/tables	Database	None	List[str] (all active table names)
GET	/api/database/tables/{table}/schema	Database	Path: table name	List[ColumnMetadataSchema]
GET	/api/database/tables/{table}/data	Database	Query: limit=50, offset=0	PaginatedDataSchema (rows, total, page)

GET	/api/dependencies/graph	Topology	None	GraphTopologySchema (nodes, edges)
GET	/api/dependencies/cycles	Topology	None	CycleReportSchema (has_cycle, cycles)
POST	/api/scanner/run	Scanner	ScanRequest (scope, table_name, rules)	ScanResultSchema (run_id, issues_found)
GET	/api/scanner/runs	Scanner	Query: limit=20	List[ScanRunSummarySchema]
GET	/api/scanner/runs/{run_id}	Scanner	Path: run_id	ScanRunDetailSchema (issues, duration)
GET	/api/issues	Triage	Query: status, severity, table	List[IntegrityIssueSchema]
GET	/api/issues/{id}	Triage	Path: issue_id	IntegrityIssueDetailSchema (raw_row, rule)
PATCH	/api/issues/{id}	Triage	IssueUpdate (status, notes)	IntegrityIssueSchema
POST	/api/repairs/preview	Repair	RepairPreviewRequest (issue_id, strategy)	RepairPreviewSchema (before, after, hash)
POST	/api/repairs/execute	Repair	RepairExecuteRequest (issue_id, hash, actor)	RepairExecutionResult (success, rows_affected)
POST	/api/transactions/simulate	ACID Lab	SimulationRequest (scenario_type)	SimulationResult (steps, verified)
GET	/api/recovery/checkpoints	Recovery	Query: table_name	List[CheckpointSchema]
POST	/api/recovery/checkpoints	Recovery	CreateCheckpointRequest (table_name, label)	CheckpointSchema (id, row_count)
POST	/api/recovery/restore	Recovery	RestoreRequest (checkpoint_id)	RestoreResultSchema (restored_rows, verified)
GET	/api/audit/logs	Compliance	Query: limit=50, event_type, actor	List[AuditEntrySchema]
GET	/api/rules	Policy	None	List[IntegrityRuleSchema]
POST	/api/rules/{id}/dry-run	Policy	Path: rule_id	DryRunResultSchema (violation_count)
POST	/api/settings/inject-demo	Lab	None	ActionStatus (injected_count, status)
POST	/api/settings/reset-baseline	Lab	None	ActionStatus (reset_count, status)

15.3 Critical Mutation Contract: Automated Scan Dispatch (`POST /api/scanner/run`)

This endpoint initiates a multi-threaded integrity scan across the relational schema. It accepts parameters specifying whether to scan all tables or target an individual relation.

Below is the exact JSON request payload and corresponding successful response structure:

```
-- Scan Trigger Request Payload (JSON)
{
  "scope": "ALL",
  "target_table": null,
  "enabled_rules": ["ORPHAN_ENROLLMENT_STUDENT", "NULL_STUDENT_EMAIL", "DOMAIN_STUDENT_AGE"]
}

-- Scan Response Payload (HTTP 200 OK)
{
  "success": true,
  "data": {
    "run_id": 42,
    "scan_type": "FULL_SCHEMA",
    "total_tables_scanned": 16,
    "total_rows_examined": 1420,
    "issues_detected": 11,
    "duration_ms": 38.45,
    "completed_at": "2026-11-03T00:15:22.104Z"
  },
  "message": "Full schema integrity scan completed successfully."
}
```

15.4 High-Safety Repair Preview Contract (`POST /api/repairs/preview`)

Prior to applying any database mutation, the frontend must request a non-destructive repair preview. The backend evaluates the repair strategy, generates the parameter-bound SQL statement, captures the before/after state, and computes the SHA-256 row-version concurrency hash.

```
-- Repair Preview Request (JSON)
{
  "issue_id": 14,
  "strategy": "SET_DEFAULT",
  "custom_value": null
}

-- Repair Preview Response with Concurrency Token (HTTP 200 OK)
{
  "success": true,
  "data": {
    "issue_id": 14,
    "target_table": "students",
    "primary_key_val": 105,
    "strategy": "SET_DEFAULT",
    "current_value": "NOT_AN_EMAIL",
    "proposed_value": "student_105@university.edu",
    "preview_sql": "UPDATE students SET email = %s, updated_at = NOW() WHERE student_id = %s",
    "blast_radius_tables": ["students", "enrollments"],
    "risk_level": "LOW",
    "expected_row_hash": "a1f9c832e65d2140bb7190e2f18392d449ac019482f34812f8490a1827498c11"
  },
  "message": "Repair preview synthesized with valid concurrency token."
}
```

15.5 Atomic Repair Execution Contract (`POST /api/repairs/execute`)

The execution contract enforces strict concurrency controls. If another concurrent process has updated the record in the interim, the SHA-256 hash recalculation fails, returning HTTP 409 Conflict and aborting the transaction.

```
-- Repair Execution Request (JSON)
{
  "issue_id": 14,
  "strategy": "SET_DEFAULT",
  "expected_row_hash": "a1f9c832e65d2140bb7190e2f18392d449ac019482f34812f8490a1827498c11",
  "actor": "ADMIN_OPERATOR"
}

-- Repair Execution Response (HTTP 200 OK)
{
  "success": true,
  "data": {
    "repair_log_id": 8,
    "issue_id": 14,
    "rows_affected": 1,
    "status": "SUCCESS",
    "restoration_token": "RESTORE_14_20261103_0015",
    "audit_event_id": 104
  },
  "message": "Atomic database repair executed successfully and committed."
}
```

Chapter 16: Automated Testing Suite & Verification Matrix

■ 10-YEAR-OLD INTUITION — AUTOMATED TESTING SUITE & VERIFICATION MATRIX

We have 45 automated robot tests! They check everything: testing if database connections work, making sure SQL injection attacks fail, testing if rollbacks actually rewind data, and verifying that preview tokens block race conditions. All tests pass with flying colors!

16.1 Testing Methodology & Fixtures

The project includes an extensive test suite built on `pytest` and Starlette's `TestClient`. Tests are organized into standalone contract validation tests (which run without a live database) and integration tests (which execute real queries against MySQL).

16.2 Test Coverage by Module

Summary of the 45 test definitions distributed across the test suite:

Test Suite File	Test Count	Core Functionality Verified
test_api_endpoints.py	14 tests	Health check, 400 validation, 404 handling, CORS origins, table routes.
test_connection.py	3 tests	SQL identifier validation allowlist, malicious injection rejection, connection diagnostics.
test_dependency_analyzer.py	3 tests	DAG node discovery, table neighborhood traversal, cascading impact assessment.
test_repair_engine.py	3 tests	Before/After previews, missing record auto-resolution, idempotent execution.
test_set_default_preview.py	6 tests	Preview validation, admin input enforcement, stale preview rejection, rollback on failure.
test_transaction_manager.py	3 tests	Atomic commit verification, simulated failure rollback count, savepoint preservation.
test_recovery_manager.py	2 tests	Snapshot checkpoint creation, table state modification, verified state restoration.
test_audit_service.py	2 tests	Sensitive credential redaction, immutable audit log insertion into database.
test_scanner.py	2 tests	Integrity scan execution, anomaly detection counts, recommendation generation.
test_unit.py	7 tests	Pure unit tests verifying schema models, regex validators, and utility helpers.

16.3 Complete Specification of All 45 Automated Test Cases

The project test harness contains 45 formal test functions validating every layer of the architecture. Below is the complete quality assurance test catalog:

#	Test Function Name	Source Test File	Scope / Assertion Validated
1	test_api_health	test_api_endpoints.py	Verifies /api/health returns 200, service name, and version.
2	test_api_cannot_manually_mark_repaired	test_api_endpoints.py	Verifies client cannot manually PATCH status to REPAIRED.
3	test_cors_trusted_origin_allowed	test_api_endpoints.py	Verifies http://localhost:5173 is accepted with credentials.

4	test_cors_untrusted_origin_rejected	test_api_endpoints.py	Verifies malicious origin is rejected by CORS middleware.
5	test_invalid_sql_identifier_returns_400	test_api_endpoints.py	Verifies SQL injection identifiers trigger HTTP 400.
6	test_api_database_status	test_api_endpoints.py	Verifies database status returns registered table counts.
7	test_api_database_tables	test_api_endpoints.py	Verifies table list contains at least 10 active schema tables.
8	test_api_list_rules	test_api_endpoints.py	Verifies rules endpoint returns at least 5 configured rules.
9	test_api_dependency_graph	test_api_endpoints.py	Verifies dependency graph returns nodes and edges arrays.
10	test_api_audit_logs	test_api_endpoints.py	Verifies audit logs endpoint returns items array.
11	test_nonexistent_scan_issues_returns_404	test_api_endpoints.py	Verifies invalid scan ID returns HTTP 404.
12	test_valid_scan_issues_regression	test_api_endpoints.py	Verifies existing scan returns HTTP 200 with issues.
13	test_nonexistent_table_neighborhood_returns_404	test_api_endpoints.py	Verifies non-existent table returns HTTP 404.
14	test_valid_table_neighborhood_regression	test_api_endpoints.py	Verifies valid table returns direct parents and children.
15	test_validate_identifier_safe	test_connection.py	Verifies safe alphanumeric table and column names pass.
16	test_validate_identifier_dangerous	test_connection.py	Verifies malicious SQL injection strings raise ValueError.
17	test_mysql_connection_diagnostics	test_connection.py	Verifies live MySQL server latency, user, and version telemetry.
18	test_dependency_graph_structure	test_dependency_analyzer.py	Verifies DiGraph contains core relational schema tables.
19	test_table_neighborhood	test_dependency_analyzer.py	Verifies neighborhood extraction returns parent/child lists.
20	test_repair_impact_assessment	test_dependency_analyzer.py	Verifies deleting parent with dependents flags CRITICAL risk.
21	test_repair_preview	test_repair_engine.py	Verifies non-destructive preview generates SQL and states.
22	test_destructive_repair_requires_confirmation	test_repair_engine.py	Verifies destructive actions enforce approval guard.
23	test_missing_record_handling_and_auto_resolution	test_repair_engine.py	Verifies preview on deleted row auto-marks issue REPAIRED.
24	test_test_a_preview_missing_customer_name	test_set_default_preview.py	Verifies missing mandatory attribute requires admin input.
25	test_test_b_preview_missing_customer_email	test_set_default_preview.py	Verifies email preview requires valid format input.
26	test_test_c_foreign_key_dependency_detection	test_set_default_preview.py	Verifies foreign key dependencies are detected before fix.
27	test_test_d_repair_failure_handling_and_rollback	test_set_default_preview.py	Verifies failure rolls back atomically and logs event.
28	test_test_e_and_f_successful_repair_and_duplicate	test_set_default_preview.py	Verifies successful repair commits and prevents dupes.
29	test_test_g_stale_preview_rejection	test_set_default_preview.py	Verifies modified row triggers StalePreviewError.
30	test_transaction_demo_commit	test_transaction_manager.py	Verifies COMMIT increases record count by exactly 1.
31	test_transaction_demo_rollback	test_transaction_manager.py	Verifies ROLLBACK restores initial count with zero leaks.
32	test_transaction_demo_savepoint	test_transaction_manager.py	Verifies SAVEPOINT rollback preserves preceding operations.

33	test_recovery_manager_class_exists	test_recovery_manager.py	Verifies RecoveryManager interface exposes required methods.
34	test_recovery_checkpoint_and_restoration	test_recovery_manager.py	Verifies full table snapshot creation and restoration.
35	test_sanitize_json_redaction	test_audit_service.py	Verifies password and token keys are replaced with [REDACTED].
36	test_audit_log_persisted	test_audit_service.py	Verifies audit events are inserted into MySQL audit_logs.
37	test_scanner_detection_run	test_scanner.py	Verifies full scan detects injected demo violations.
38	test_recommendation_generation	test_scanner.py	Verifies scanner generates structured healing proposals.
39	test_unit_rule_model_validation	test_unit.py	Verifies Pydantic rule schema validation and type parsing.
40	test_unit_issue_severity_enums	test_unit.py	Verifies severity enum values strictly match allowed types.
41	test_unit_email_regex_positive	test_unit.py	Verifies RFC-compliant email strings pass regex filter.
42	test_unit_email_regex_negative	test_unit.py	Verifies invalid email formats are correctly rejected.
43	test_unit_row_token_deterministic	test_unit.py	Verifies identical row states produce identical SHA-256 tokens.
44	test_unit_row_token_sensitivity	test_unit.py	Verifies 1-character modification alters SHA-256 token.
45	test_unit_audit_redaction_clean	test_unit.py	Verifies non-sensitive dictionaries remain untouched.

Chapter 17: Local Deployment, Setup Wizard & Administration Guide

■ 10-YEAR-OLD INTUITION — LOCAL DEPLOYMENT, SETUP WIZARD & ADMINISTRATION GUIDE

Getting the project running is as easy as 1-2-3! We wrote an automated 1-click Python setup script that checks your MySQL, creates the database, builds all 16 tables, and loads the data automatically so anyone can run it!

17.1 System Hardware & Software Prerequisites

Before initiating local deployment, verify that the host machine satisfies the following engineering baseline specifications:

1. Database Engine: MySQL Community Server 8.0+ or MariaDB 10.4+ (e.g., standard XAMPP or standalone installer) running on port 3306.

2. Backend Runtime: Python 3.10, 3.11, 3.12, or 3.13 (with `pip` and standard development headers).

3. Frontend Toolchain: Node.js 18.0+ or 20.0+ LTS along with `npm` (Node Package Manager).

4. Operating System: Windows 10/11, macOS Sonoma/Sequoia, or Ubuntu Linux 22.04/24.04 LTS.

17.2 The 1-Click Automated Setup Wizard (`setup_database.py`)

To eliminate manual configuration errors and ensure that even non-technical evaluators can bootstrap the entire system in under 60 seconds, the project includes an automated setup script: `setup_database.py`.

When executed via `python setup_database.py`, the script performs the following sequential automation pipeline:

Step 1: Environment Discovery: Inspects `backend/.env` for host, user, password, and port settings. If absent, it queries default local root configurations.

Step 2: Database Provisioning: Connects to the MySQL instance and executes `CREATE DATABASE IF NOT EXISTS self_healing_test_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci`.

Step 3: Sequential SQL Script Execution: In topological foreign-key order, the wizard reads and executes:

- `01_create_database.sql`: Provisions all 16 InnoDB relational and telemetry tables.

- `02_seed_baseline_data.sql`: Populates valid academic entities (departments, instructors, courses, students, enrollments).

- `03_seed_rules.sql`: Registers the 16 active integrity validation rules.

- `04_create_audit_triggers.sql`: Compiles native MySQL `BEFORE UPDATE` and `BEFORE DELETE` immutability triggers.

- `05_demo_violations.sql`: Creates sandbox staging anomalies for immediate testing.

Step 4: Verification Check: Queries `INFORMATION_SCHEMA.TABLES` to ensure all 16 tables exist and reports total record counts.

17.3 Manual Step-by-Step Command Line Deployment Guide

For developers who prefer granular terminal control, execute the following commands in separate shells to launch the application stack:

```
# Terminal 1: Backend Setup & Server Launch
cd self-healing-relational-database-integrity-system
python -m venv venv
.\venv\Scripts\activate            # On macOS/Linux: source venv/bin/activate
pip install -r backend/requirements.txt
python setup_database.py           # Automated schema & data initialization
uvicorn backend.main:app --reload --port 8000

# Terminal 2: Frontend Setup & Dev Server Launch
cd self-healing-relational-database-integrity-system/frontend
npm install
npm run dev
# Open browser at http://localhost:5173
```

17.4 Comprehensive Diagnostic & Troubleshooting Reference Matrix

The matrix below outlines common development environment issues, diagnostic checks, and proven remediation procedures:

Diagnostic Symptom	Probable Root Cause	Investigative Command	Remediation Procedure
MySQL Connection Refused (10061)	MySQL daemon is stopped or port blocked.	netstat -ano | findstr 3306	Start MySQL via XAMPP Control Panel or 'net start MySQL'.
Access Denied for 'root'@'localhost'	MySQL root has a configured password.	Check C:/xampp/phpMyAdmin/config.inc.php	Set DB_PASSWORD in backend/.env to match local credential.
ModuleNotFoundError: pymysql	Virtualenv missing required Python packages.	pip list | findstr pymysql	Execute 'pip install -r backend/requirements.txt'.
Vite: ECONNREFUSED :8000	FastAPI backend server is not running.	curl http://localhost:8000/api/health	Launch backend via 'uvicorn backend.main:app --reload --port 8000'.
Vite Port 5173 In Use	Previous node process still holds port.	netstat -ano | findstr 5173	Terminate orphan process via 'taskkill /PID /F' or use --port 5174.

Chapter 18: Oral Exam & Viva Defense Master Preparation (30 In-Depth Q&As;)

■ 10-YEAR-OLD INTUITION — ORAL EXAM & VIVA DEFENSE MASTER PREPARATION (30 IN-DEPTH Q&AS;)

Here are the top 30 questions professors love to ask during exams! We give you crystal-clear, rock-solid answers so you can explain every single detail with confidence.

18.1 1. System Purpose & Architecture

Q1: What is the primary purpose of this project and what problem does it solve?
The project implements an autonomous, human-supervised, transaction-safe database integrity detection, dependency analysis, self-healing, and audit traceability engine for MySQL InnoDB. It addresses the reality that enterprise databases regularly suffer from data degradation caused by bulk ETL imports, partial staging runs, accidental administrator edits, and unconstrained operational schemas. It bridges the gap between passive databases and active integrity management without using unpredictable machine learning.

Q2: How does this project differ from a standard CRUD application?
A standard CRUD application simply performs point inserts, reads, updates, and deletes against predefined forms. It treats the database as a passive dumb store. This project is a Meta-Diagnostic Engine operating above the business data layer. It inspects schema structure dynamically via INFORMATION_SCHEMA, constructs directed dependency graphs using NetworkX to calculate cascading blast radiuses, generates non-destructive repair previews with cryptographic row-version tokens, and executes repairs inside genuine ACID rollback-guaranteed transactions.

Q3: What are the three architectural tiers of the system?
The system is organized into: 1) Presentation Tier (React 18, Vite 5, Tailwind CSS, Recharts) providing 11 dashboard views; 2) Application Logic Tier (Python 3.13, FastAPI, Uvicorn, PyMySQL, NetworkX) orchestrating detection, graph analysis, preview synthesis, and transaction execution; and 3) Data Persistence Tier (MySQL InnoDB via local XAMPP) managing 16 relational and engine tables with full foreign keys, check bounds, and immutability triggers.

Q4: Why was MySQL InnoDB chosen as the database storage engine instead of MyISAM?
InnoDB provides genuine ACID transaction compliance, row-level locking (SELECT ... FOR UPDATE), foreign key referential integrity enforcement, and write-ahead redo logging. In contrast, older engines like MyISAM only support table-level locking, have no foreign key enforcement, and cannot execute ROLLBACK statements, making transactional self-healing impossible.

Q5: What role does PyMySQL play in the backend architecture?
PyMySQL is a pure-Python MySQL client driver. It allows FastAPI services to execute raw, high-performance parameterized SQL queries, inspect cursor metadata, manage explicit multi-statement transactions (autocommit=False), and handle native database exceptions directly without the abstraction overhead of heavy Object-Relational Mappers (ORMs).

18.2 2. Relational Schema & Anomaly Detection

Q6: Why is there an unconstrained staging table (`staging_enrollments`) in the database?
In real-world enterprise databases (such as universities or financial institutions), external batch data imported from legacy files or third-party APIs regularly contains errors. If physical foreign keys were enforced directly on staging tables, the database engine would reject the entire batch file on the first bad record. Having an unconstrained staging table allows raw data to land safely, enabling our Integrity Scanner to inspect, isolate, and heal records prior to final promotion into production tables.

Q7: What is an orphan record, and how is it detected mathematically?
An orphan record is a child row containing a non-NULL foreign key pointing to a primary key value that does not exist in the referenced parent table. In relational algebra, it is defined as: O = { r in Child | r.fk != NULL and not exists p in Parent with p.pk = r.fk }. The scanner detects orphans using a relational anti-join: `SELECT c.* FROM child c LEFT JOIN parent p ON c.fk = p.pk WHERE p.pk IS NULL AND c.fk IS NOT NULL`.

Q8: Why is a LEFT JOIN used for orphan detection instead of an INNER JOIN?
An INNER JOIN discards non-matching rows and only returns records where parent and child both exist. A LEFT JOIN preserves all child rows regardless of parent existence. When a parent record is missing, the LEFT JOIN populates all parent columns with NULL. This allows the predicate `WHERE p.pk IS NULL` to cleanly isolate orphan records.

Q9: How does duplicate candidate key detection work?
Duplicate detection evaluates logical entity uniqueness across unconstrained tables. It executes a `GROUP BY` query on the candidate key columns (e.g. `student_id, course_id`) with `HAVING COUNT(*) > 1`. The engine sorts the conflicting primary keys ascending, designates the lowest ID as the canonical record, and generates violation issues for all subsequent duplicate IDs.

Q10: What is the difference between a physical NULL and an empty string breach?
A physical database NULL represents the complete absence of a value. An empty string (`''`) or whitespace string (`' '`) is physically non-NULL in SQL but semantically violates business rules for mandatory attributes (such as student names). Our scanner checks both using the condition `WHERE col IS NULL OR TRIM(col) = ''`.

Q11: How are domain range violations detected?
Domain range checks evaluate numeric columns against parameterized business boundaries defined in `integrity_rules.parameters_json`. For example, grade marks must satisfy `min_value: 0.00` and `max_value: 100.00`. The scanner executes `WHERE val < min_value OR val > max_value` to detect out-of-bounds anomalies.

Q12: How are domain enumeration violations detected?
Enumeration checks enforce finite set membership for categorical columns. For staging enrollment status, allowed values are `['ENROLLED', 'COMPLETED', 'DROPPED']`. The scanner evaluates `WHERE status NOT IN ('ENROLLED', 'COMPLETED', 'DROPPED')` to detect invalid status strings like `'SUSPENDED'` or `'MAGIC'`.

Q13: How does the scanner prevent duplicate issues during re-scans?
The scanner implements deduplication hashing based on a unique composite key: `(table_name, record_id, column_name, category)`. If an issue matching this key is already recorded in `integrity_issues` with status `OPEN`, the scanner skips re-insertion, preventing catalog clutter.

18.3 3. Graph Theory, Dependencies & Blast Radius

Q14: Why is the NetworkX library used in this DBMS project?
NetworkX models relational table dependencies as a Directed Graph G = (V, E). Nodes V represent tables, and directed edges E represent foreign key relationships pointing from parent tables to child tables. This enables computational graph algorithms such as cycle detection, topological sorting, and transitive reachability analysis.

Q15: What is a circular dependency, and why is it dangerous in databases?
A circular dependency (cycle) occurs when a chain of foreign keys loops back on itself (Table A references Table B, which references Table C, which references Table A). Cycles make it impossible to insert or delete rows cleanly without constraint violations and can cause catastrophic locking deadlocks during cascading operations. The system detects cycles using `nx.simple_cycles(G)`.

Q16: What is a Relational Blast Radius?
The Relational Blast Radius defines the maximum theoretical boundary of downstream data that could be corrupted, altered, or orphaned if a modification or deletion is applied to a target table. It is mathematically calculated as the transitive closure of all reachable descendant nodes: BlastRadius(T) = Descendants(G, T).

Q17: How does the system calculate live referential child counts?
Beyond schema-level graph reachability, the `DependencyAnalyzer` executes live SQL queries against all direct physical child tables, issuing `SELECT COUNT(*) WHERE fk_col = %s` for the specific target primary key. This reveals exactly how many actual live records are attached to the record being repaired.

Q18: What is topological sorting and how does it assist in safe database operations?
Topological sorting linearly orders directed graph vertices such that for every directed edge (u, v), vertex u comes before v. In database migrations, reverse topological sorting provides the exact safe deletion order: leaf child records are detached or deleted before parent records are touched, preventing foreign key constraint violations.

18.4 4. Concurrency, Previews & Repair Execution

Q19: What is a Stale Preview Hazard (Race Condition) in database repairs?
A Stale Preview Hazard occurs in multi-user database environments when an administrator reviews a repair preview for Record #8, but another process alters or deletes Record #8 before the administrator clicks 'Execute'. If executed blindly, the repair would overwrite newly modified data.

Q20: How does the cryptographic Row-Version Token prevent stale previews?
During preview generation, the engine normalizes the target row into a sorted dictionary and computes a SHA-256 hash token: `Token = SHA256(JSON(CanonicalRow))`. Before executing the repair inside the transaction, the engine acquires an exclusive lock and re-computes the hash. If the hashes differ, the engine aborts execution immediately with a `StalePreviewError`.

Q21: What is the purpose of `SELECT ... FOR UPDATE` during repair execution?
`SELECT ... FOR UPDATE` acquires an exclusive pessimistic row-level write lock in MySQL InnoDB. It ensures that no other concurrent transaction can read, modify, or delete the target record while our healing transaction is verifying tokens and applying fixes.

Q22: How does the `QUARANTINE` repair strategy work?
`QUARANTINE` is our safest data removal strategy. Instead of permanently deleting a corrupted record, the engine serializes the entire row into structured JSON, inserts it into `quarantine_records` along with the source

table, primary key, issue type, reason, and operator identity, and only then deletes the row from the source table. This guarantees zero data loss.

Q23: How does the `SET_DEFAULT` repair strategy handle mandatory missing values?
For missing mandatory attributes (like a student's name), `SET_DEFAULT` strictly prohibits fabricated fake placeholders like '[PROPOSED: Default]'. The preview flags `requires_admin_input: True`, and the engine forces the human administrator to enter an authoritative legal replacement value before the repair can be executed.

Q24: How does the `MAP_PARENT` repair strategy resolve orphan records?
`MAP_PARENT` re-links an orphan child record to an existing, valid parent primary key. The engine verifies that the target parent ID actually exists in the parent table, and then issues an `UPDATE child SET fk_col = %s WHERE id = %s` inside an atomic transaction.

18.5 5. Transactions, Recovery & Auditing

Q25: What does ACID stand for and how is each property enforced in this project?
ACID stands for Atomicity (guaranteed by explicit BEGIN, COMMIT, and automatic ROLLBACK on failure), Consistency (guaranteed by foreign keys, check bounds, and pre-repair verification), Isolation (guaranteed by InnoDB REPEATABLE READ isolation and row-level write locks), and Durability (guaranteed by InnoDB write-ahead redo logs after commit).

Q26: What is a SAVEPOINT in MySQL transactions?
A SAVEPOINT is a bookmark created within an active transaction using `SAVEPOINT savepoint_name`. It allows the application to execute a partial rollback (`ROLLBACK TO SAVEPOINT savepoint_name`) undoing only operations executed after the bookmark while preserving all preceding operations within the same transaction.

Q27: How does application-level checkpoint recovery work?
Before high-risk repairs, the `RecoveryManager` captures table DDL and serializes all current table rows into JSON, storing the snapshot in `recovery_checkpoints`. During restoration, the engine verifies primary keys, checks foreign key bounds, upserts snapshot rows, and deletes newly added post-snapshot rows inside a transaction, mathematically verifying that post-restore row counts exactly match snapshot row counts.

Q28: How do native database triggers enforce audit log immutability?
The SQL schema installs `BEFORE UPDATE` and `BEFORE DELETE` triggers on `audit_logs` and `transaction_logs`. If any user or application attempts to update or delete a log entry, the trigger executes `SIGNAL SQLSTATE '45000'` with a critical error, causing MySQL to reject the operation physically at the engine level.

Q29: How does the audit redaction filter protect sensitive credentials?
Before logging event payloads to `audit_logs`, the `AuditService` sanitizes dictionaries. Any key containing substrings like `password`, `secret`, `token`, `pwd`, `hash`, or `credit_card` has its value replaced with the mask `'[REDACTED]'`, preventing credential leaks in persistent logs.

Q30: Why is this system described as a 'human-supervised self-healing system' rather than fully autonomous?
In relational database engineering, fully autonomous remediation without human oversight is dangerous because business intent cannot be inferred by an algorithm alone. For example, if a student's name is missing, only a human can supply the real name. The system autonomously detects, models dependencies, classifies

risks, and generates previews, but requires human approval for high-risk operations to ensure maximum safety.

Chapter 19: Technical Limitations, Security Hardening & Future Scope

■ 10-YEAR-OLD INTUITION — TECHNICAL LIMITATIONS, SECURITY HARDENING & FUTURE SCOPE

Every honest engineer talks about what can be improved! In future versions, we can add user logins with passwords, support distributed cloud databases, and add AI suggestions to recommend realistic replacement values for missing data.

19.1 Current Technical Boundaries

A rigorous engineering evaluation acknowledges current system limitations:

1. Connection Pooling: The current PyMySQL driver creates connections per operation rather than managing a persistent pooled connection pool.

2. Single-Tenant Scope: The system is designed for single-database instances; cross-database distributed integrity across sharded clusters is not implemented.

3. Strategy Coverage: Strategies `RESTORE_PARENT` and `MERGE_DUPLICATE` are declared in the enumeration catalog but require manual human execution in version 2.4.0.

19.2 Security Hardening Roadmap

• Role-Based Access Control (RBAC): Integrate JWT authentication and role-based permissions (Admin, Operator, Auditor).

• Recursive Audit Redaction: Expand the shallow dictionary redaction sanitizer to recursively inspect deeply nested JSON lists and objects.

19.3 Future Research: Distributed Graph Consensus & Cross-Shard Integrity

In hyperscale cloud architectures (such as Google Cloud Spanner, AWS Aurora, or Vitess), tables are frequently sharded across multiple physical nodes. In such distributed topologies, evaluating a global referential Directed Acyclic Graph requires distributed consensus algorithms.

Future iterations of this system could implement a Two-Phase Commit (2PC) or Raft-backed graph consensus engine where local shard agents compute partitioned subgraphs and coordinate with a distributed metadata sequencer to calculate cross-shard blast radiuses with zero distributed deadlocks.

Chapter 20: Appendix, Project Glossary & File Manifest

■ 10-YEAR-OLD INTUITION — APPENDIX, PROJECT GLOSSARY & FILE MANIFEST

Here is the master dictionary and map of every single file in the project! Whenever you want to know which Python file or SQL script does what, check this page.

20.1 Technical Glossary

• ACID: Atomicity, Consistency, Isolation, Durability — the four fundamental properties of transactional databases.

• Anti-Join: A relational query pattern (`LEFT JOIN ... WHERE parent.pk IS NULL`) used to isolate unreferenced child records.

• Directed Acyclic Graph (DAG): A directed graph with no closed loops, used to represent valid relational dependency hierarchies.

• Immutability: An architectural property guaranteeing that logged records cannot be edited or erased once created.

20.2 Source Code Repository Manifest

Directory structure and file responsibilities in Release 2.4.0:

File Path	Module Layer	Primary Technical Responsibility
backend/app/main.py	FastAPI App	Application entry point, CORS middleware, router registration.
backend/app/services/integrity_scanner.py	Scanner Service	Dynamic metadata querying, rule evaluation, issue persistence.
backend/app/services/dependency_analyzer.py	Graph Service	NetworkX DAG modeling, cycle detection, blast radius assessment.
backend/app/services/repair_engine.py	Healing Service	Non-destructive previews, SHA-256 tokens, ACID repair execution.
backend/app/services/transaction_manager.py	ACID Simulator	Commit, rollback, and savepoint demonstration workflows.
backend/app/services/recovery_manager.py	Recovery Service	Table snapshot checkpointing, row restoration, state verification.
backend/app/services/audit_service.py	Audit Service	Event logging, sensitive credential redaction, trigger integration.
setup_database.py	Setup Wizard	Automated 1-click database, table, rule, and demo violation provisioner.
database/01-05 SQL Scripts	SQL Schema	Database creation, InnoDB tables, sample baseline, rules, demo violations.

20.3 Formal Mathematical Notation & Relational Calculus Guide

Below is the reference glossary of formal mathematical symbols, relational algebra operators, and graph theory notation employed throughout this report:

Symbol / Notation	Mathematical Definition	Context in Relational Integrity System
G = (V, E)	Directed Graph with Vertices V and Edges E	V represents tables; E represents foreign key references.
O	Orphan Record Set	Child rows with non-NULL FK lacking matching parent PK.
D	Candidate Key Duplicate Group	Group of records sharing identical logical candidate keys.
BlastRadius(T)	Transitive Reachable Descendants	Set of all downstream dependent tables affected by modifying T.
SHA-256(Row)	Cryptographic Hash Function	Deterministic row-version signature for optimistic concurrency control.
P -> C	Referential Constraint Mapping	Arrow pointing from parent table P to referencing child table C.
sigma_{cond}(R)	Relational Selection Operator	Filtering rows matching violation criteria (WHERE clause).
pi_{cols}(R)	Relational Projection Operator	Selecting specific columns for inspection or preview (SELECT clause).
R bowtie_{cond} S	Relational Theta Join	Joining child and parent relations along referential attributes.
Delta_{rollback}	Transaction Undo Delta	Reverting uncommitted database state changes upon exception.